

Project Title:

E-Commerce & Inventory Tracking

Problem statement:

An online store struggles to track customer orders and manage stock availability. They need a Salesforce solution that integrates with e-commerce platforms, manages customer orders, updates inventory automatically, and provides reports on best-selling products.

Implementation:

Phase 1: Problem Understanding & Industry Analysis

➤ Requirement Gathering:

- Track products, orders, customers, and inventory levels.
- Automate stock reduction & low-stock alert.
- Provide dashboards for sales & inventory insights.

➤ Stakeholder Analysis:

- Customers (place orders).
- Sales Team (manage orders).
- Inventory Manager (track stock).
- System Administrator -manages users, roles, configurations.
- Business Owner/Management - views reports, dashboards, KPIs.
- Finance/Accounts Team - tracks payments, refunds, invoices.

➤ **Business Process Mapping:**

- Customer selects product → Order created → Inventory auto-updates → Confirmation sent → Reports and dashboards generated.

➤ **Industry Use Case:**

- Similar to Amazon/Flipkart stock tracking.

➤ **AppExchange Exploration:**

- Look for prebuilt inventory/order management apps as references.

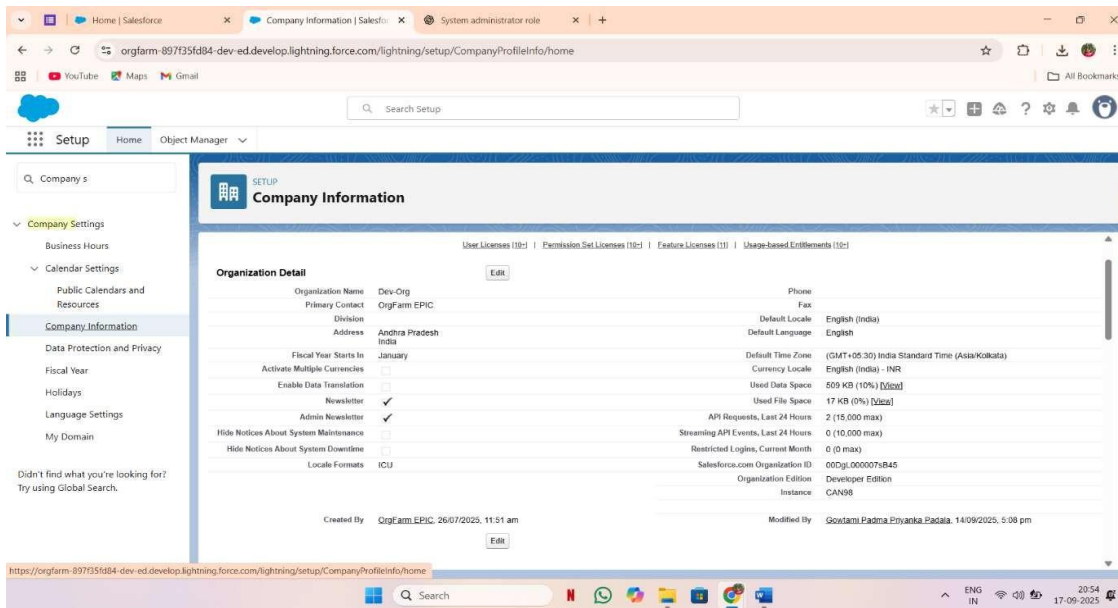
Phase 2 — Org Setup & Configuration:

➤ **Setup salesforce Org:**

- Sign up for a **free Salesforce Developer Edition** (developer.salesforce.com).
- Choose **Sales Cloud** as the edition.

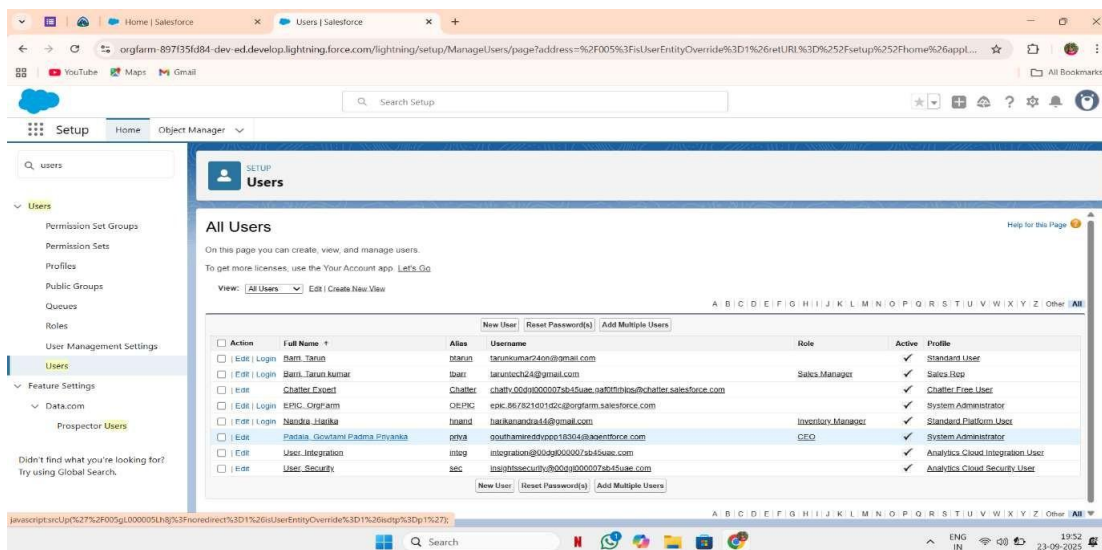
➤ **Configure Company Settings:**

- **Company Info** → Go to *Setup* → **Company Settings** → **Company Information** → Fill in company name, address, currency, time zone.
- **Business Hours** → *Setup* → **Business Hours** → Create working hours (e.g., Mon–Fri 9 AM – 5 PM).
- **Holidays** → *Setup* → **Holidays** → Add public holidays (these can be used in escalation rules).

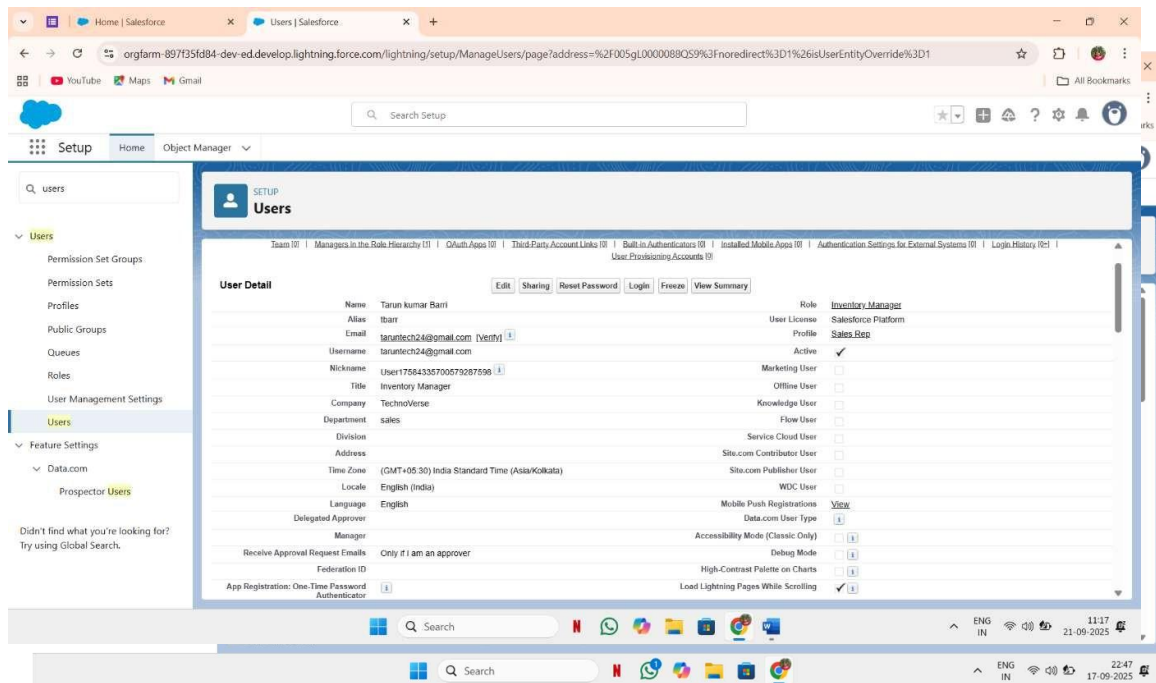


➤ Create Users, Roles, Profiles

- Users → Setup → Users → New User
 - Example: Sales Rep, Inventory Manager, System Admin.
- Roles → Setup → Roles → Set up Roles
 - Example hierarchy:
 - CEO/Admin (top)
 - Sales Manager
 - Inventory Manager



- **Profiles → Setup → Profiles**
 - Use *System Administrator* profile for admin.
 - Clone *Standard User* profile to create *Sales Rep Profile* and *Inventory Manager Profile* with appropriate permissions.



➤ Define Permission Sets

- **Setup → Permission Sets → New**
 - **Order Approvers** → Add “Modify All” on Orders.
 - **Inventory Editors** → Add “Edit” access on Inventory objects.
- Assign these permission sets to users who need special access without changing their profile.

➤ Configure Sharing Settings:

- **Organization-Wide Defaults (OWD):**

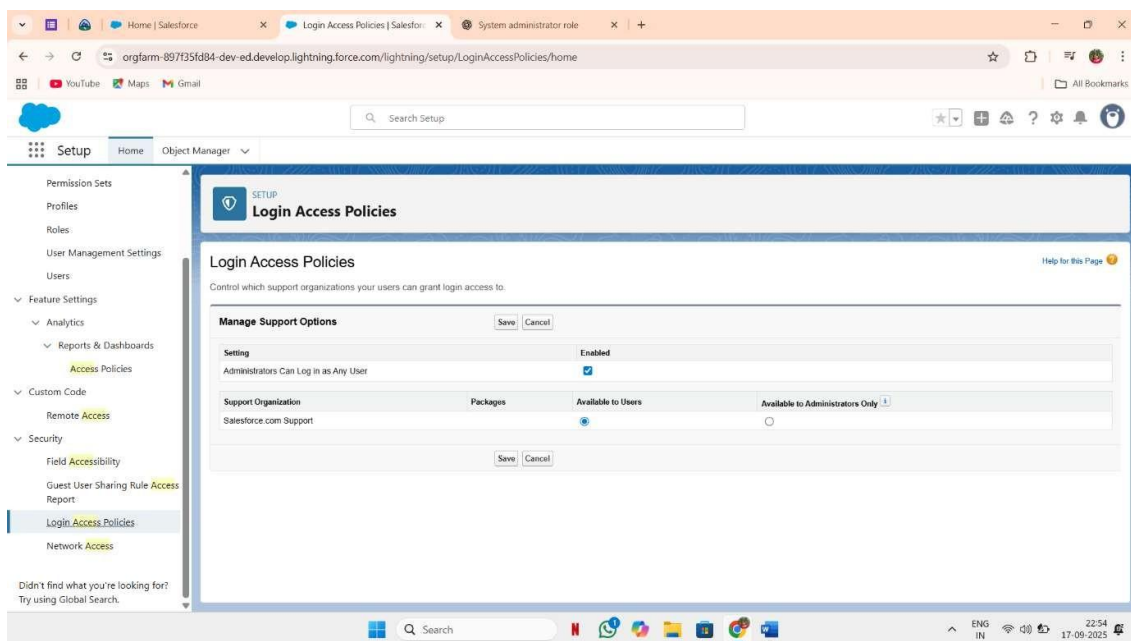
Setup → **Sharing Settings** → Set *Orders* = *Private* (only owner sees records).

- **Sharing Rules:**

- Example: Create a rule to share *Orders* with *Sales Managers* so they see all team orders.
- Use these as required by the process.

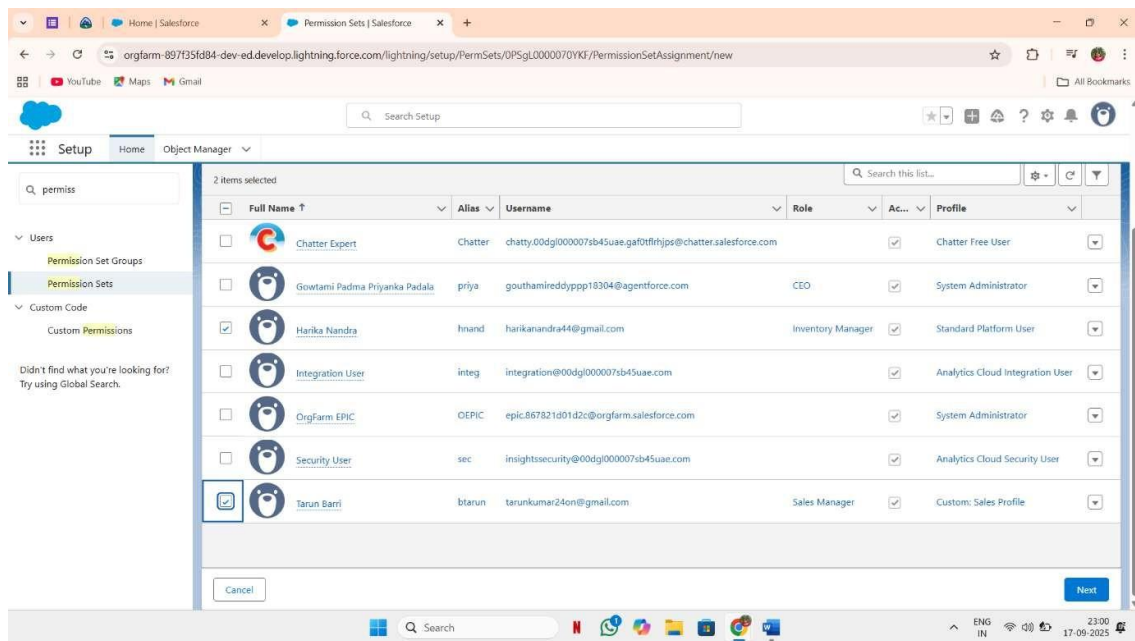
➤ **Enable Security Settings**

- **Login Access Policies** → Allow admins to log in as users for troubleshooting.



➤ **MFA (Multi-Factor Authentication):**

- Enable from *Setup* → **Identity Verification** → **Multi-Factor Authentication**.
- Adding multi factor authentication for users using permission sets



➤ Audit Trail:

- Setup → View Setup Audit Trail → Monitor configuration changes

Phase 3: Data Modeling & Relationships

➤ Identify Core Business Entities

- E-Commerce Objects:
 - Product (standard: Product2)
 - Order / OrderItem
 - Customer (standard: Account & Contact)
 - Cart (custom object if required before checkout)
 - Payment (custom object)
- Inventory Objects:
 - Inventory Item (custom object linked to Product2)
 - Warehouse / Location (custom object)
 - Stock Movement (custom object for tracking stock-in/stock-out)
 - Supplier / Vendor (custom object)

➤ Create Standard & Custom Objects

- Use Object Manager in Salesforce Setup.
- Extend standard objects (Account, Contact, Product, Order).
- Create custom objects like:
 - Warehouse__c
 - Inventory_Item__c
 - Payment__c

A. Standard Objects to Leverage

1. **Account** – Represents **customers**

- Example: *Amazon Seller, Customer John Doe.*

2. **Contact** – Represents people tied to Accounts.

- Example: *Customer's shipping contact info.*

3. **Product2** – Represents your products.

- Example: *iPhone 15, Samsung TV.*

4. **Order / OrderItem** – Captures purchase details.

- Example: *Order #1234 with 3 OrderItems.*

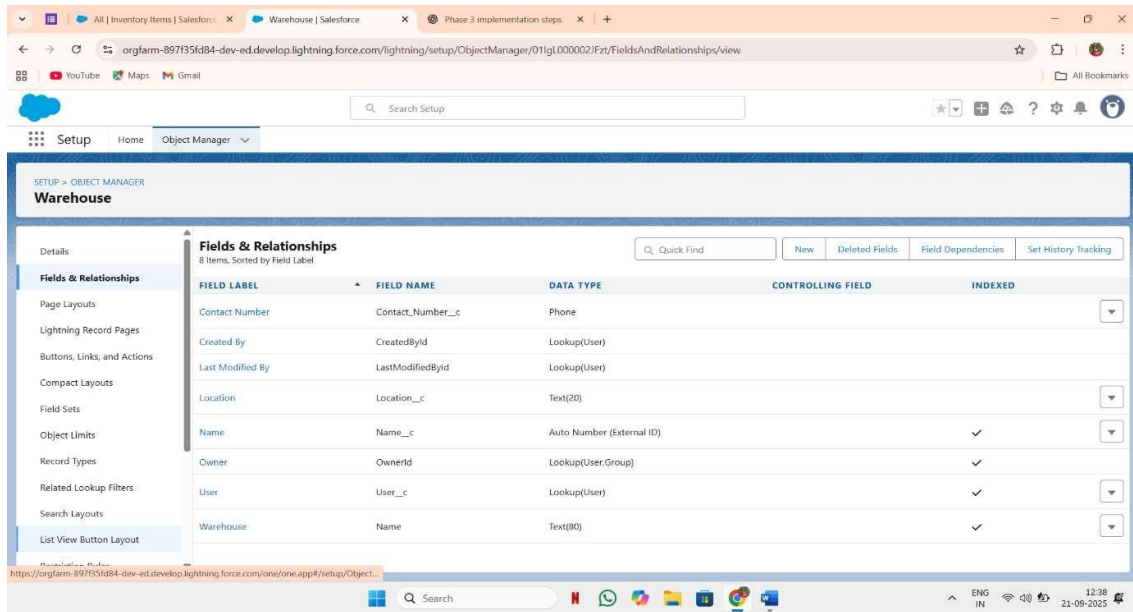
B. Custom Objects to Create

- Create tabs for these custom objects so that they are visible to the application.

1. **Warehouse__c**

- Stores warehouse locations.
- Key Fields:
 - Name (Auto-number: WH-0001, WH-0002...)
 - Location__c (Text/Geolocation)
 - Manager__c (Lookup → User)

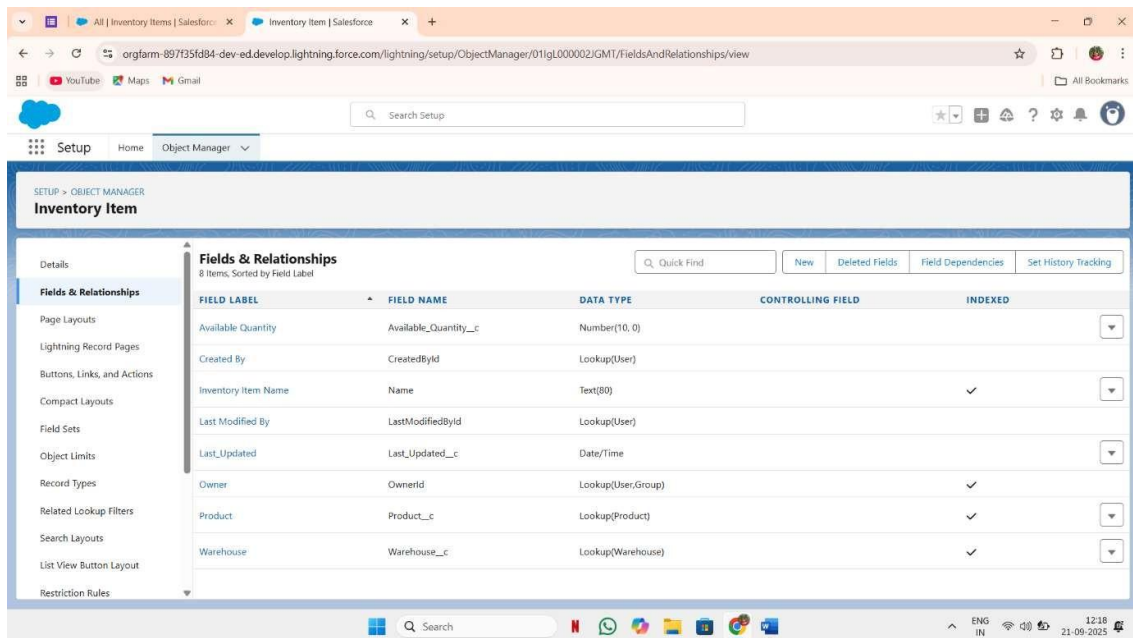
- Contact Number(number)



2. Inventory_Item__c

- Tracks stock for each product in each warehouse.
- Key Fields:
 - Product__c (Lookup → Product)
 - Warehouse__c (Lookup → Warehouse__c)
 - Available_Quantity__c (Number)
 - Reorder_Level__c (Number)

- Last_Updated__c (Date/Time)



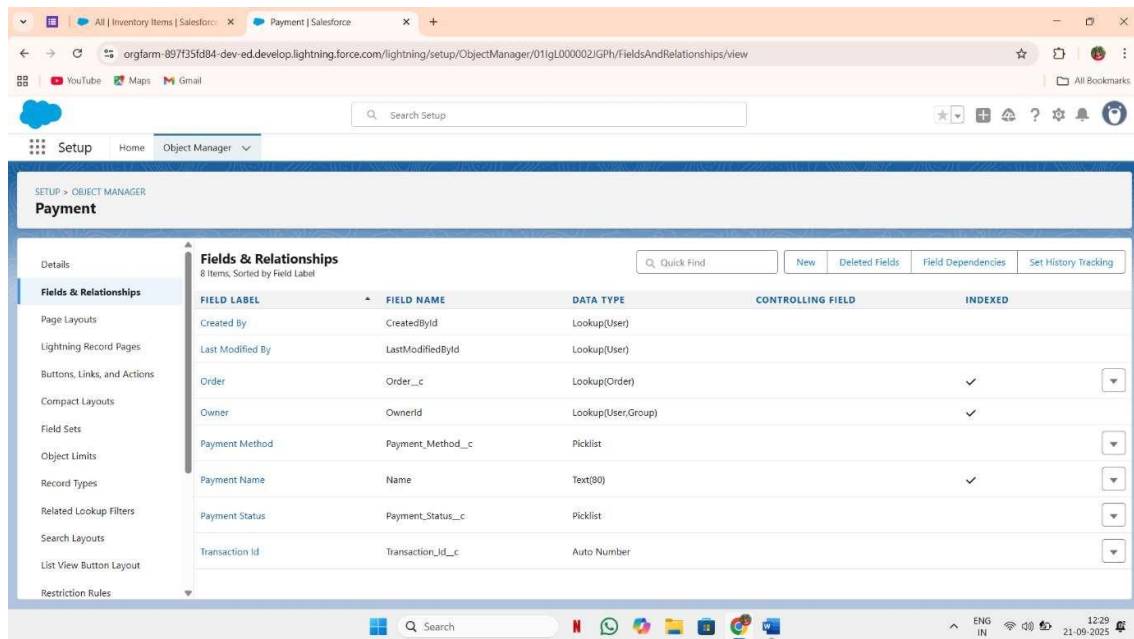
The screenshot shows the Salesforce Object Manager interface for the 'Inventory Item' object. The 'Fields & Relationships' tab is selected, displaying a list of 8 fields. The fields are sorted by Field Label. The table includes columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Available Quantity	Available_Quantity__c	Number(16, 0)		
Created By	CreatedById	Lookup(User)		
Inventory Item Name	Name	Text(80)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Last_Updated	Last_Updated__c	Date/Time		
Owner	OwnerId	Lookup(User:Group)		✓
Product	Product__c	Lookup(Product)		✓
Warehouse	Warehouse__c	Lookup(Warehouse)		✓

4. Payment__c

- Captures payment details for an order.
- Key Fields:
 - Order__c (Lookup → Order)
 - Payment_Method__c (Picklist: Credit Card, UPI, Net Banking, COD)
 - Payment_Status__c (Picklist: Pending, Completed, Failed)

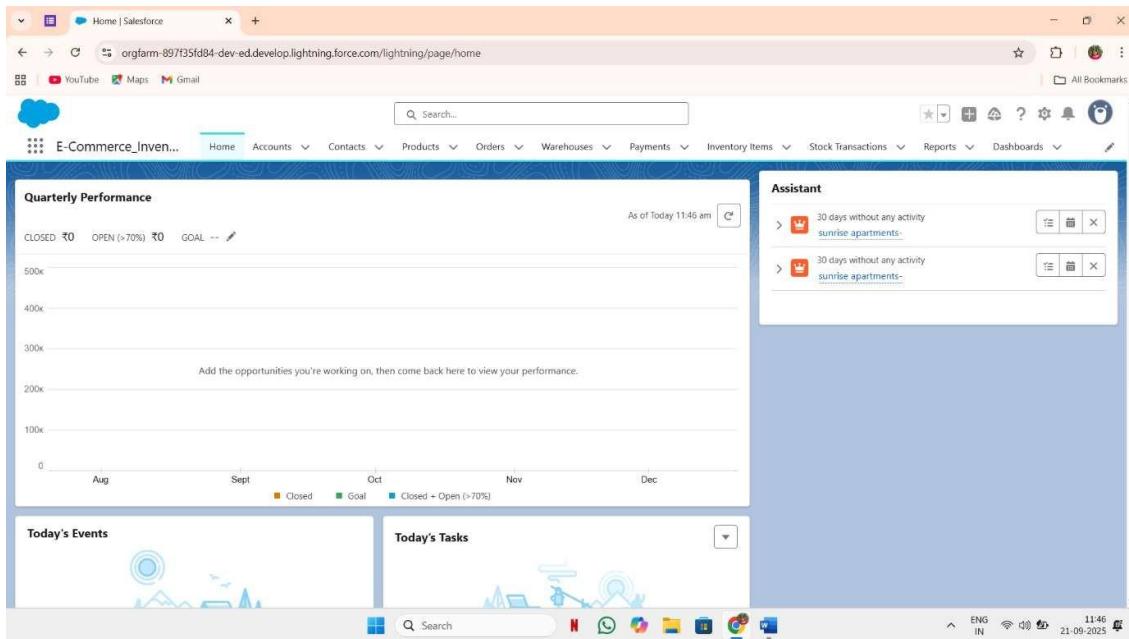
- Transaction_Id__c (Text)



➤ Create the Lightning App (Setup → App Manager → New Lightning App)

- App Label: **ECommerceInventory**
- App Name: ECommerceInventory
- Description: E-commerce + Inventory Management
- Branding: Set logo & color
- Navigation Style: Console or Standard (Console recommended for Inventory Manager)
- Add Navigation Items (tabs): Accounts, Contacts, Products, Price Books, Orders, Payments, Warehouse, Inventory Item, Stock Transaction, Cart, Reports, Dashboards.

- **Assign Visibility:** Choose which profiles can see the app (Sales Rep, Inventory Manager, Admin)

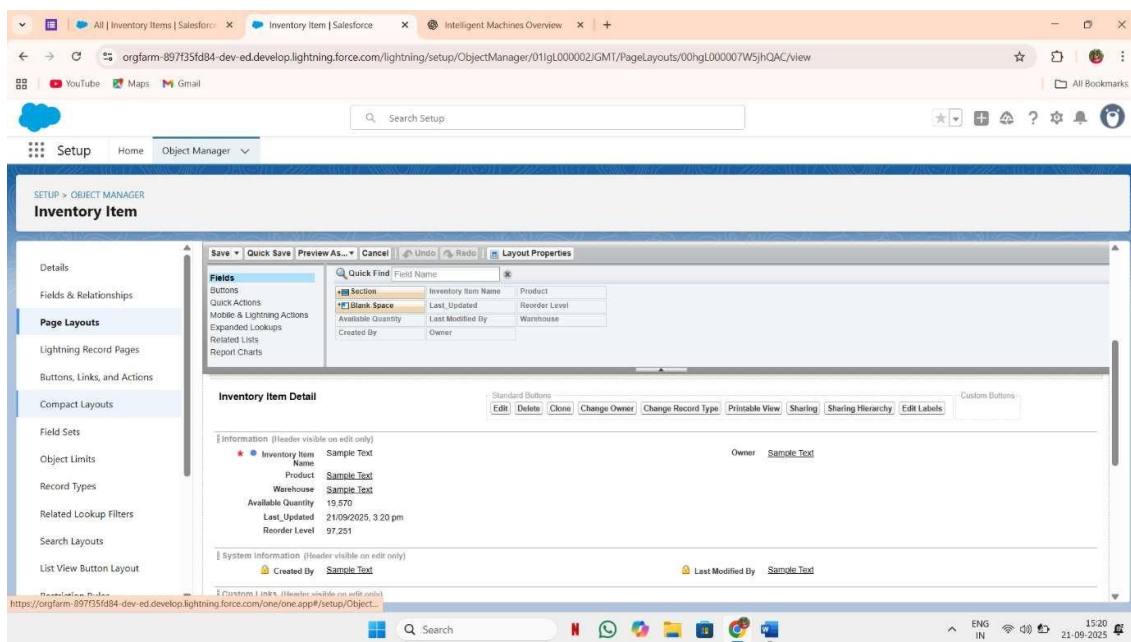
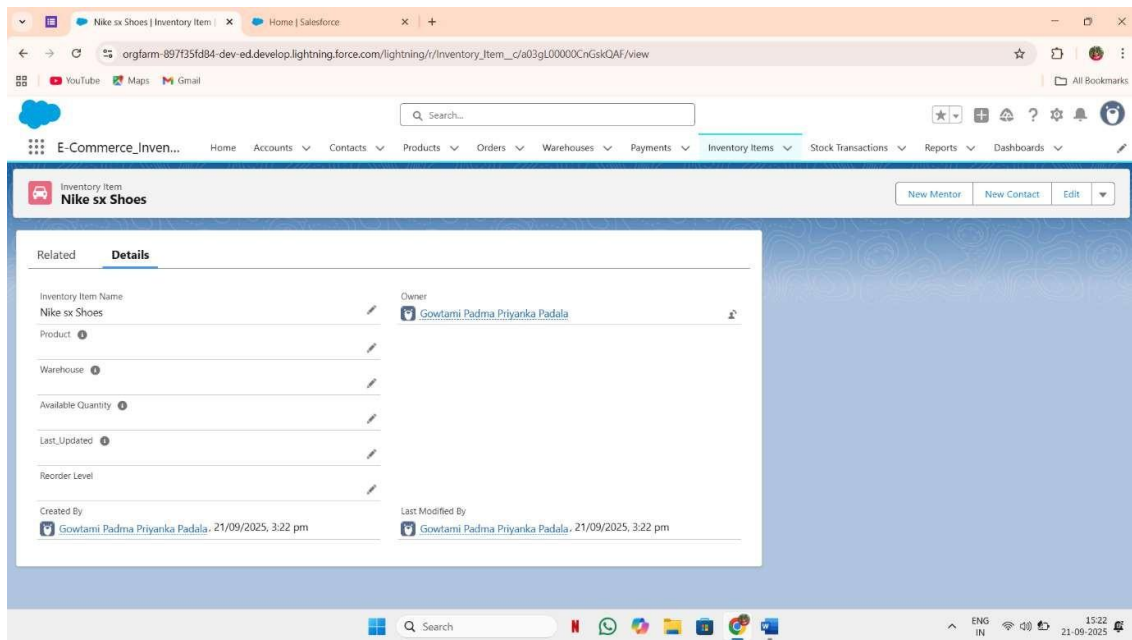


➤ Set Up Record Types

- **Purpose:** Let you create **different business processes, picklist values, and page layouts** for the same object.
- For Products:
 - Record Types: *Electronics, Clothing, Grocery* (different layouts/fields).
- For Orders:
 - Record Types: *Retail Order, Bulk Order*.
- Think of Record Types as “categories” of records within the same object.

➤ Design Page Layouts & Compact Layouts

- **Purpose:** Control the **fields, sections, related lists, and buttons** visible on a record's detail page.



- Customize layouts for each profile (Sales Rep, Inventory Manager).
- Example:

- Inventory Manager sees stock fields (Reorder Level, Warehouse).
- Sales Rep sees customer/order-related fields.
- Assign **different page layouts** to profiles (or record types).
- Think of Page Layouts as “what fields and sections you see and edit.”
- Think of Compact Layouts as “the quick info you see at the top or on mobile.”

➤ **Establish Relationships:**

• Relationship	• Field Type
• Account → Contact	• Lookup / Master-Detail (often master-detail)
• Account → Order	• Lookup
• Product → Inventory Item	• Lookup
• Warehouse → Inventory Item	• Lookup
• Order → Payment	• Lookup

➤ Use Junction Objects

- If needed, create junction objects:
 - Example: *Product-Warehouse Mapping* if a product is stored in multiple warehouses.
 - Example: *Customer-Cart* for handling multiple customers adding items to different carts.

➤ External Objects

- If inventory or payment data resides in an external ERP/Payment system, set up External Objects via Salesforce Connect.

Phase 4: Process Automation (Admin)

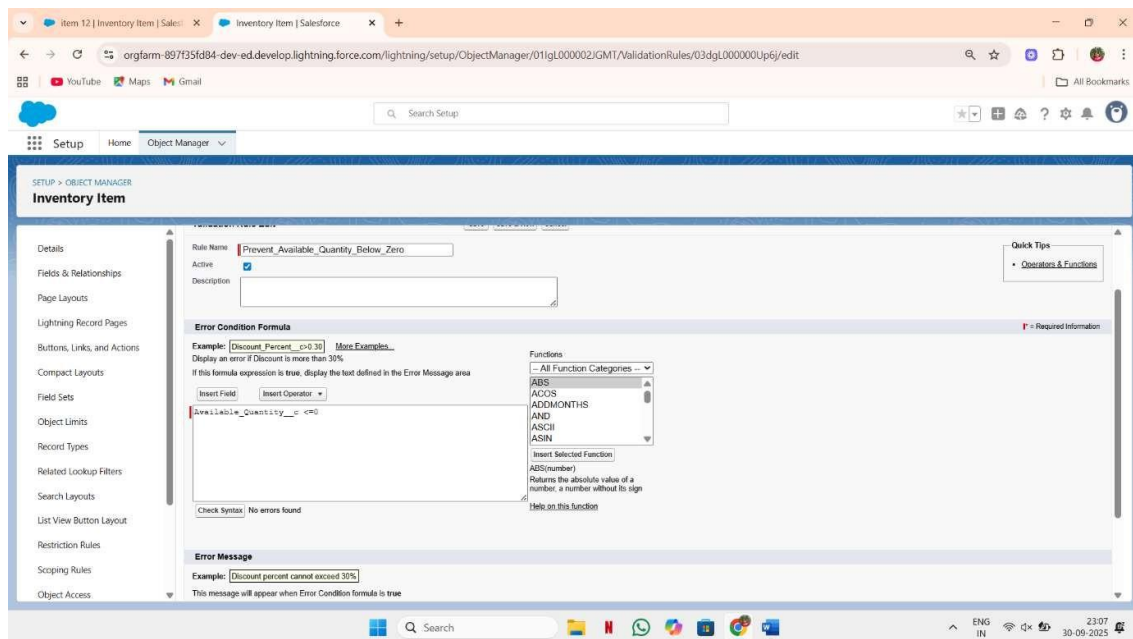
This phase focuses on reducing manual work and ensuring business processes run efficiently using Salesforce automation tools.

➤ Validation Rules

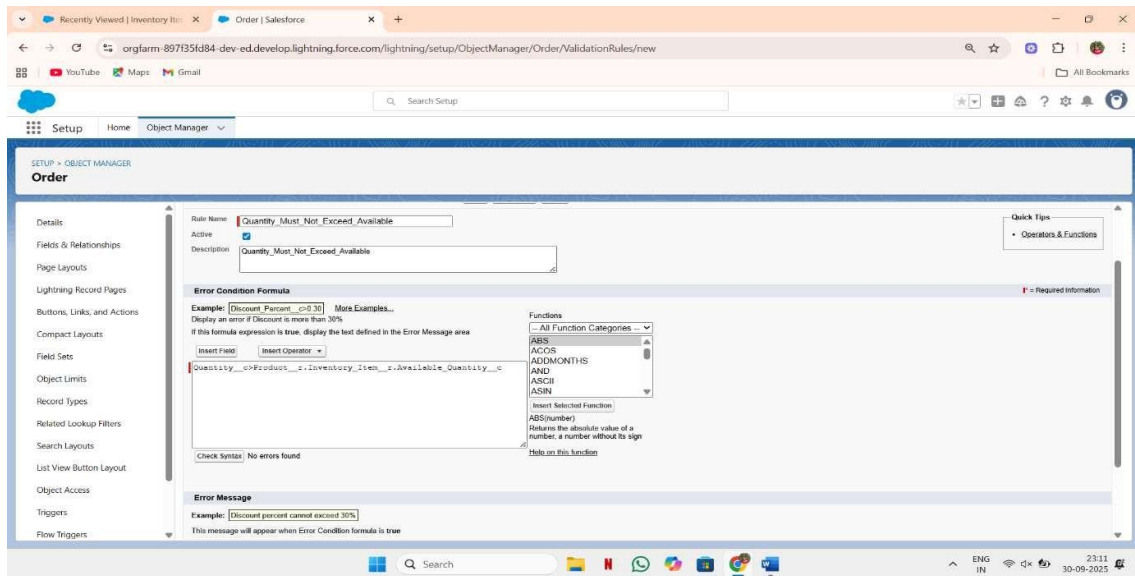
- Enforce data quality by restricting invalid entries.
- Go to Setup → Object Manager → [Your Object, e.g., Product or Inventory] → Fields & Relationships.
- Click “New” → Validation Rule.

Use cases:

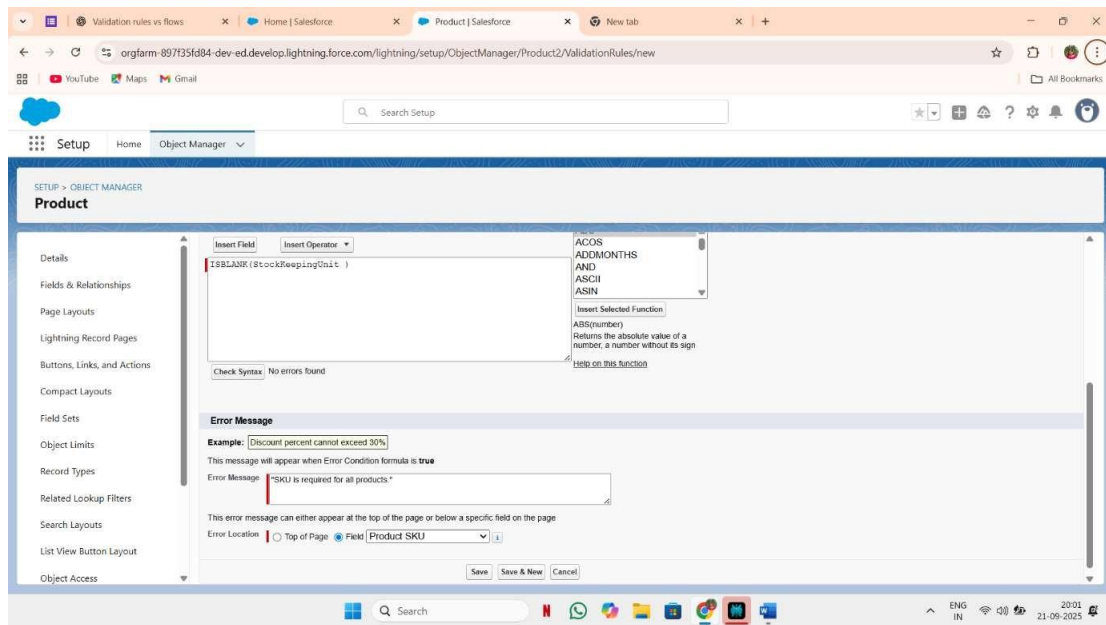
- Inventory Control:
 - Prevent Available_Quantity__c from going below zero.



- Prevent Quantity__c from being greater than available stock in inventory.



- Product Compliance:
- SKU must be filled in for all products.



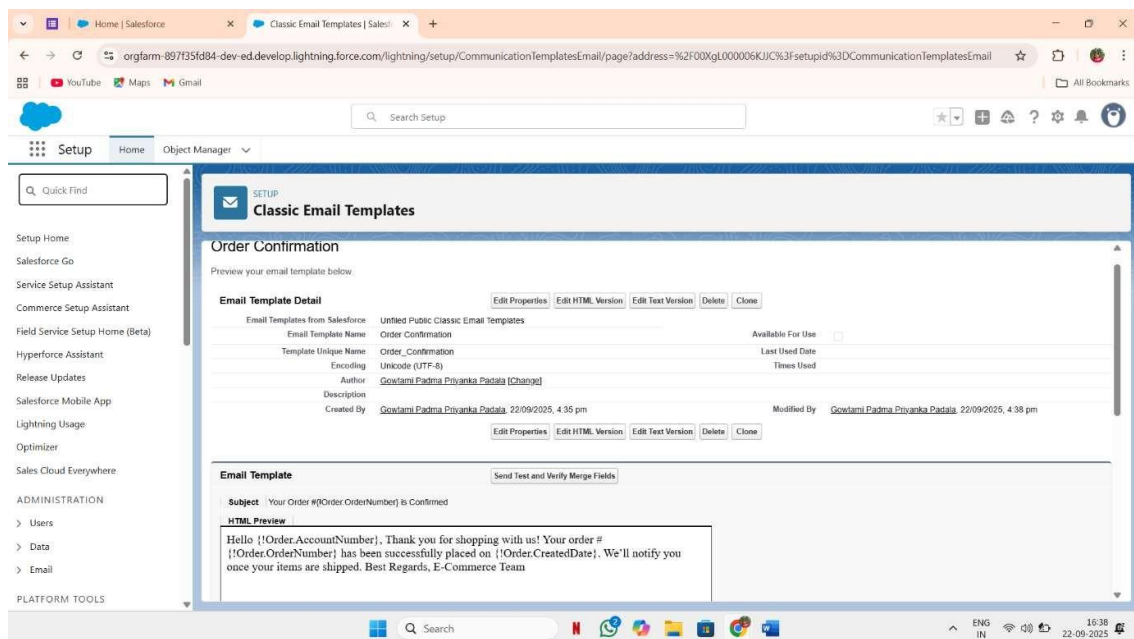
➤ Workflow Rules

- Automate simple if/then actions (though being replaced by Flow).

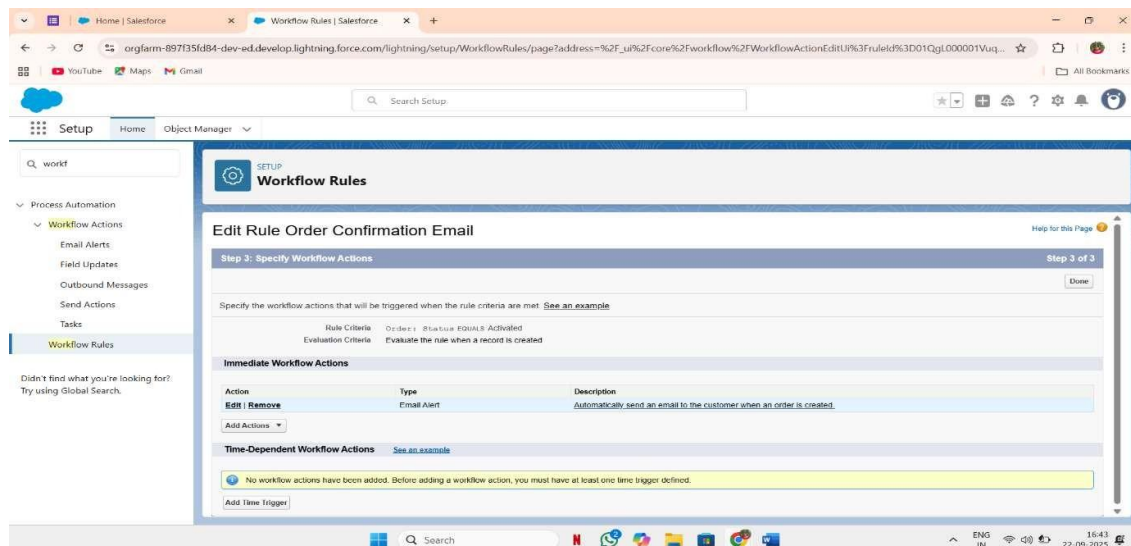
Order Confirmation Email → Automatically send an email to the customer when an order is created.

- Order Confirmation Email → Automatically send an email to the customer when an order is created.

Create email templates:



Create workflow rule:



➤ Process Builder

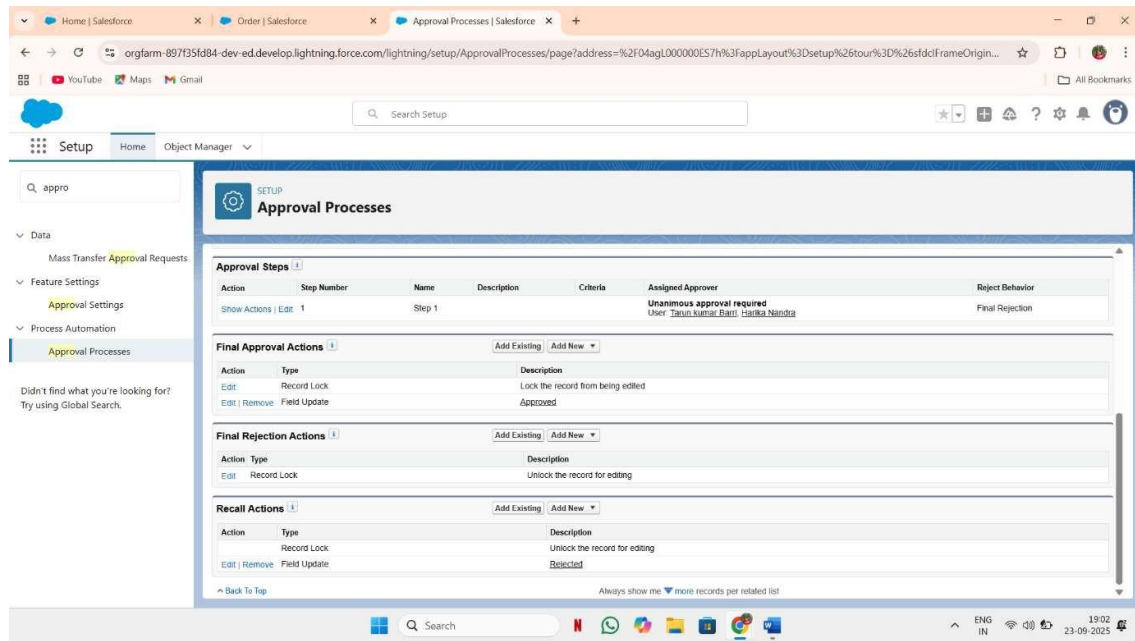
- Automate multi-step logic (being replaced by Flow gradually).

➤ Approval Process

- Route records for managerial approval.
- I made an Approval Process for your E-commerce + Inventory Management app with the use case:

"If an Order Amount is high (say > ₹10,00,000), it must be approved by a manager before processing."

- created an approval process on the **Order** object using Jump Start Wizard where the entry criteria is TotalOrderPrice > 100000. Added fields like Approval_Status__c (Pending, Approved, Rejected) and set the approvers as sales manager and inventory manager. Configured actions so that when submitted, the record is locked and an email is sent; when approved, update the status to Approved, unlock the record, and notify the sales rep; and when rejected, update the status to Rejected and notify the rep.

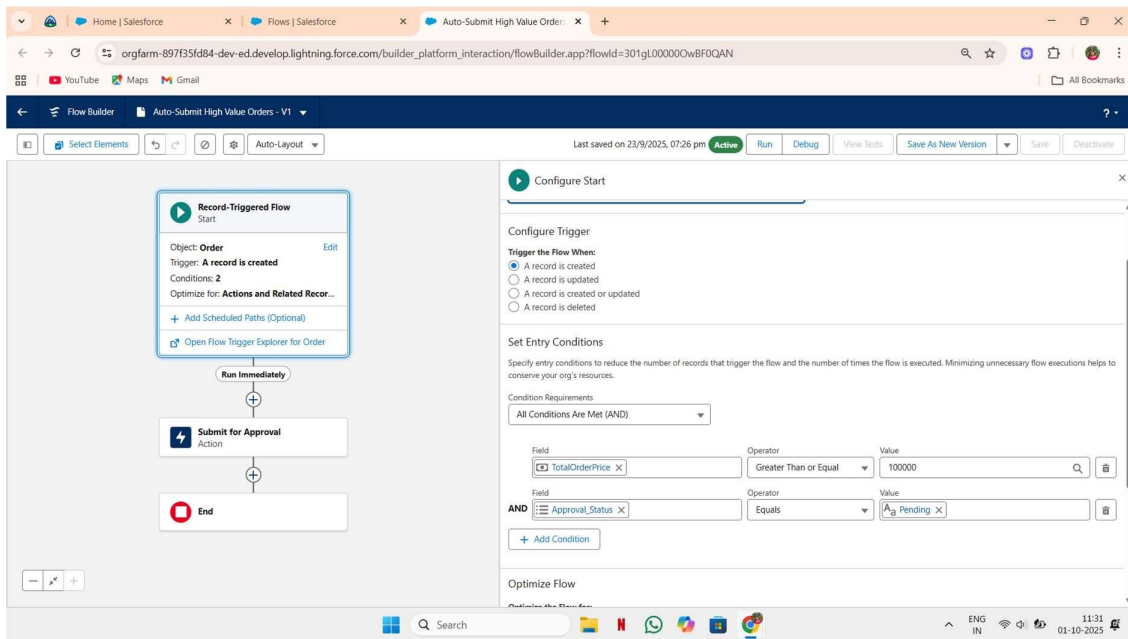


➤ Flow Builder

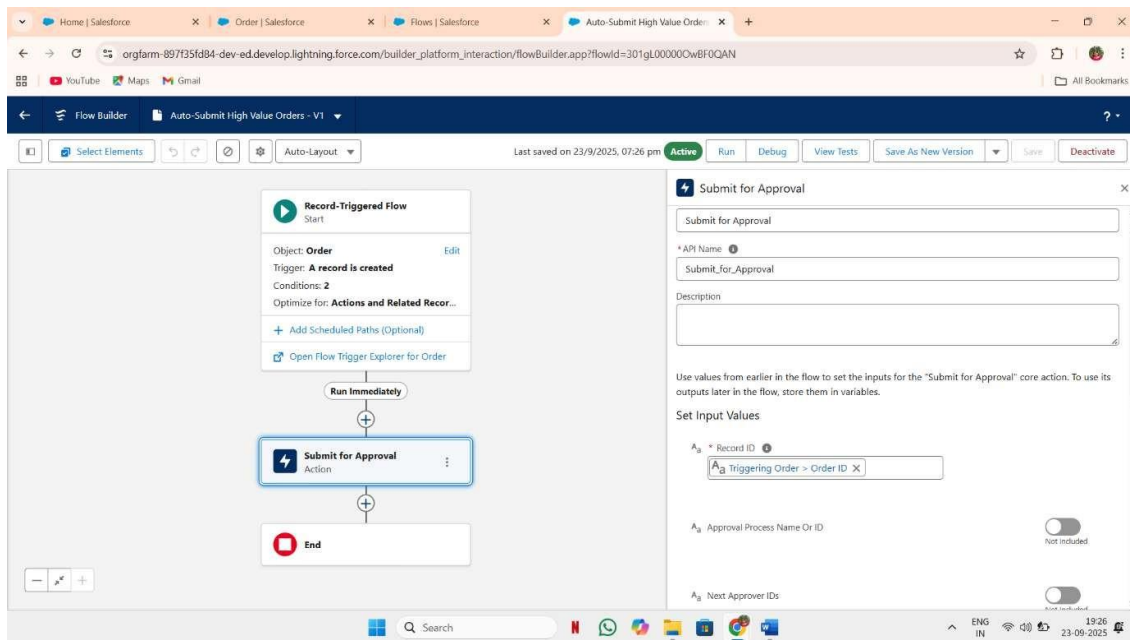
- **Record-Triggered Flow:** Auto-update fields when conditions are met.
- **Screen Flow:** Guided UI for data entry (e.g., Case resolution steps).
- **Scheduled Flow:** Automate time-based actions ().
 - **Auto-launched Flow:** Execute behind the scenes without user interaction.

Now I created a record-triggered flow to automate the above approval process.

Whenever an order is created or updated with totalorderprice > 100000 and status Pending, the **Flow automatically submits it into the approval process**, the record gets locked, and the assigned manager/approver receives the request without the user manually clicking *Submit for Approval*.

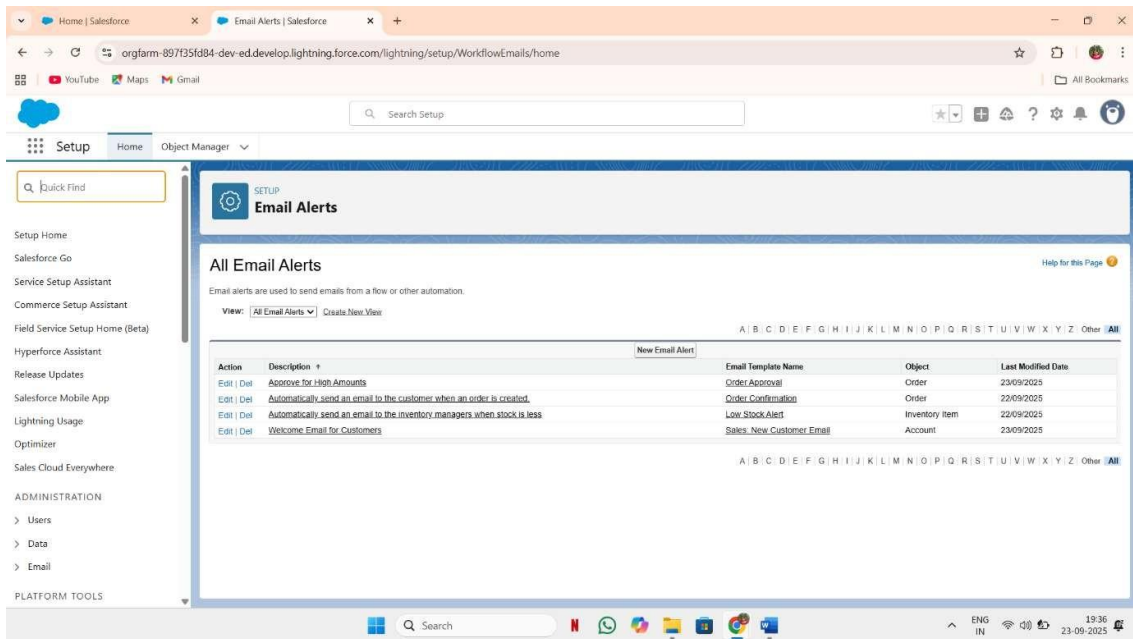


Activate the account:

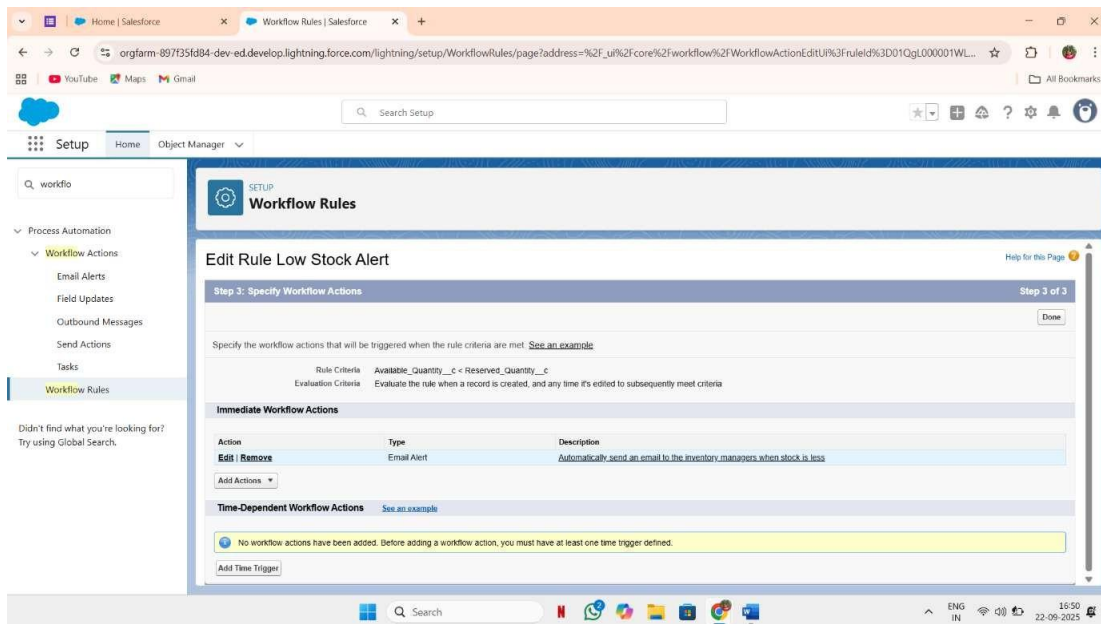


➤ Email Alerts

- Send templated emails triggered by workflow/process/flow.



- Order Status Update Notification → Notify the sales team or customer when the order status changes (e.g., Pending → Shipped).
- Low Stock Alert → Send an email or notification to inventory managers if `Available_Quantity__c < Reserved_Quantity__c`.



➤ Field Updates

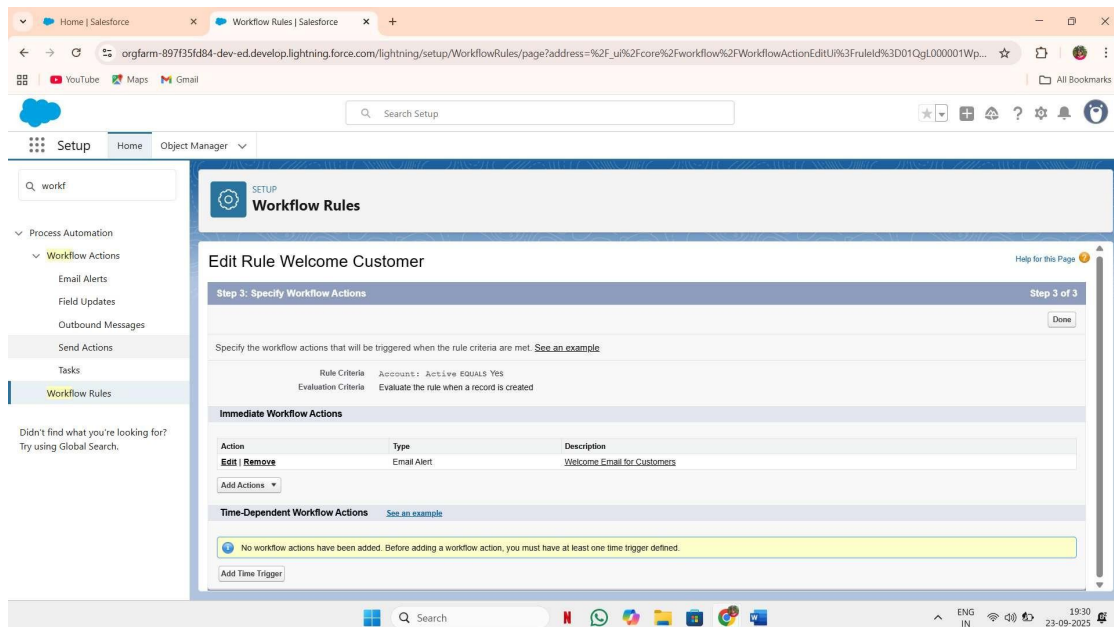
- Auto-update field values based on logic. -> Triggered by flows

➤ Tasks

- Create automatic tasks for users. Also done using flows.
- Follow-up Task → Automatically create a task for a sales rep when a customer places their first order.
- Pending Order Reminder → Create tasks if an order has been pending for too long.

➤ Custom Notifications

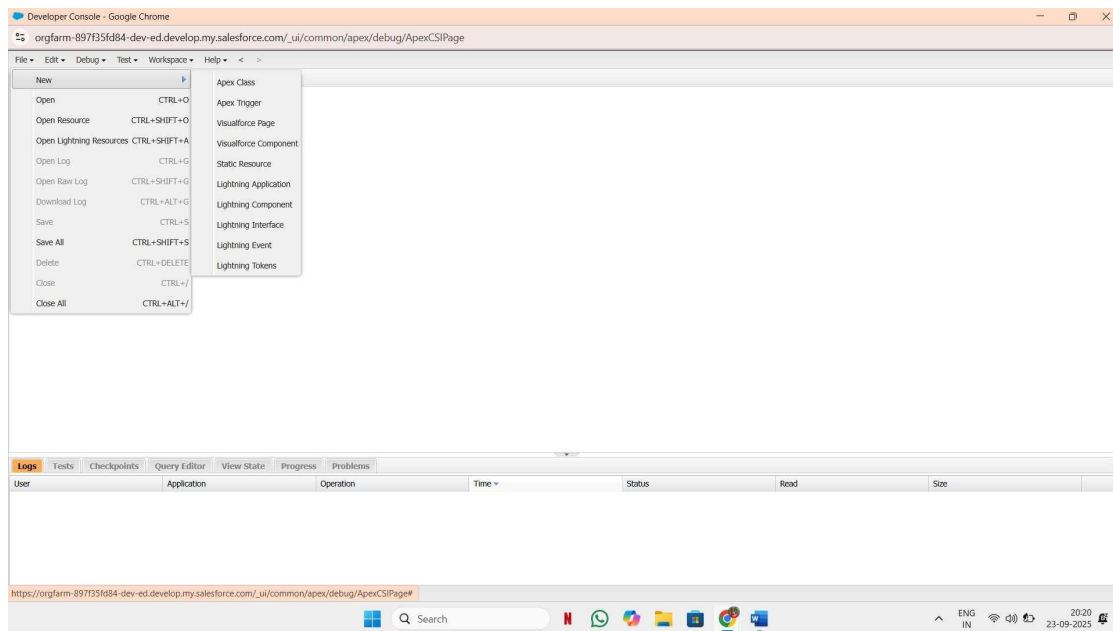
send emails to customers when they created an account in the app.



Phase 5: Apex Programming (Developer)

In this phase I worked with the developer console. Where we write apex code to perform actions, triggers etc.

From setup-> developer console



1. Classes & Objects

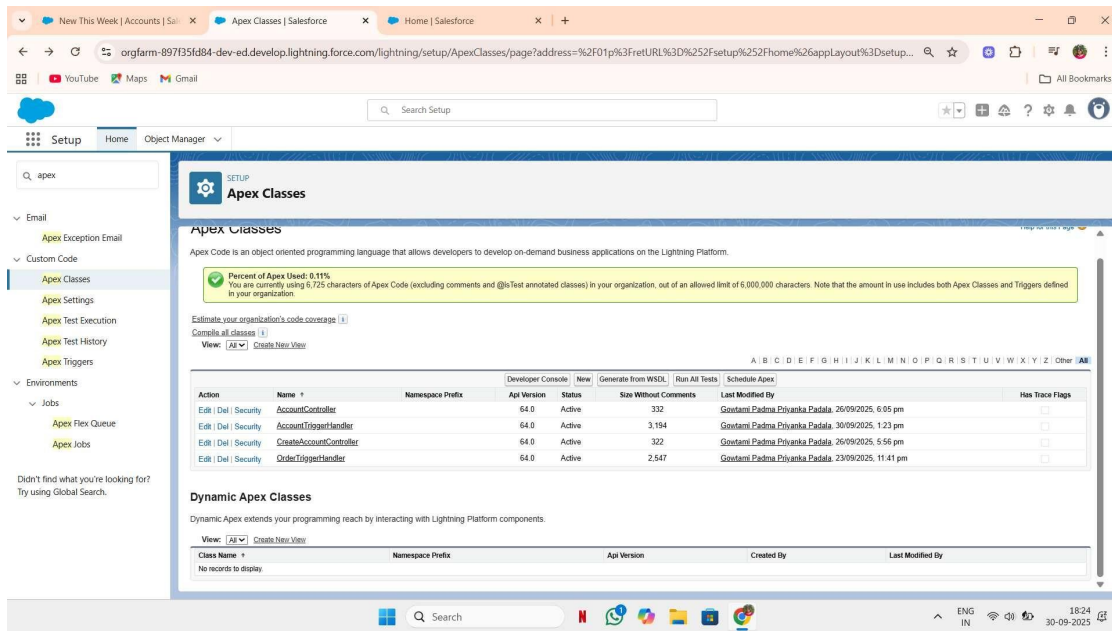
We use these apex classes and sObjects for referencing the original salesforce records.

For example:

Account-> for referencing account records.

Contacts->for contact records

Inventory Item-> for inventory records



2. Apex Triggers (before/after insert/update/delete)

- **Apex Triggers are pieces of code that execute automatically before or after records are inserted, updated, deleted, or undeleted.**

Example:

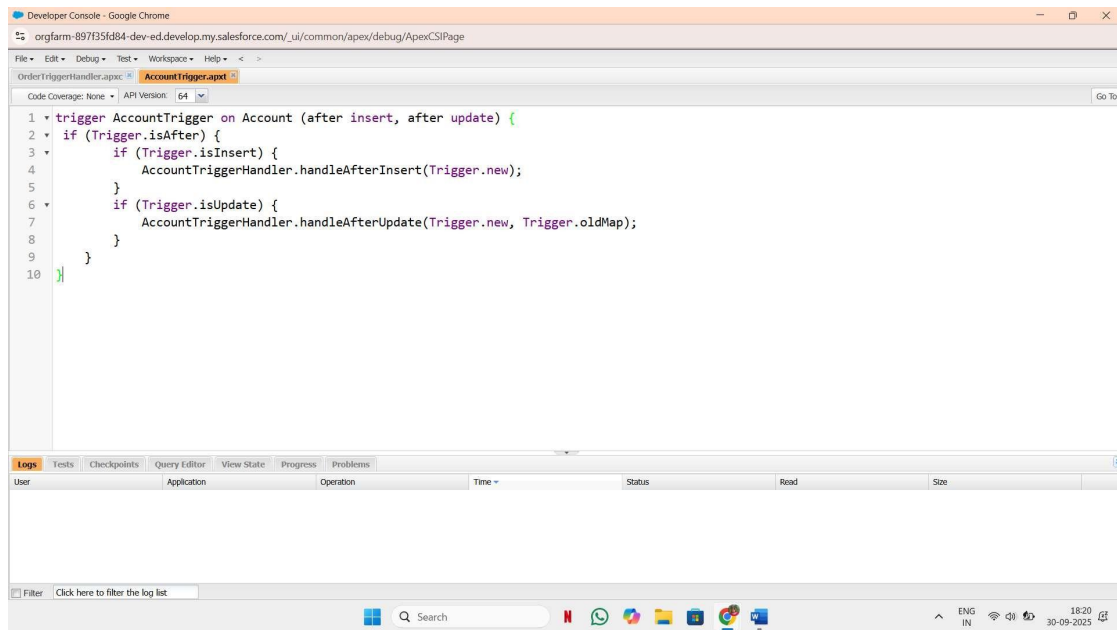
- **Before Insert:** Automatically assign a unique SKU to new products.
- **After Insert:** Send a welcome email to customers after account creation.
- **Before Update:** Prevent inventory quantity from going negative.
- **After Update:** Update related records, like adjusting stock levels after an order is shipped.
- **Before Delete:** Prevent deletion of products that are part of active orders.

For this project I applied the following triggers: Firstly I want to create and update contacts whenever an account is created or updated.

Requirement:

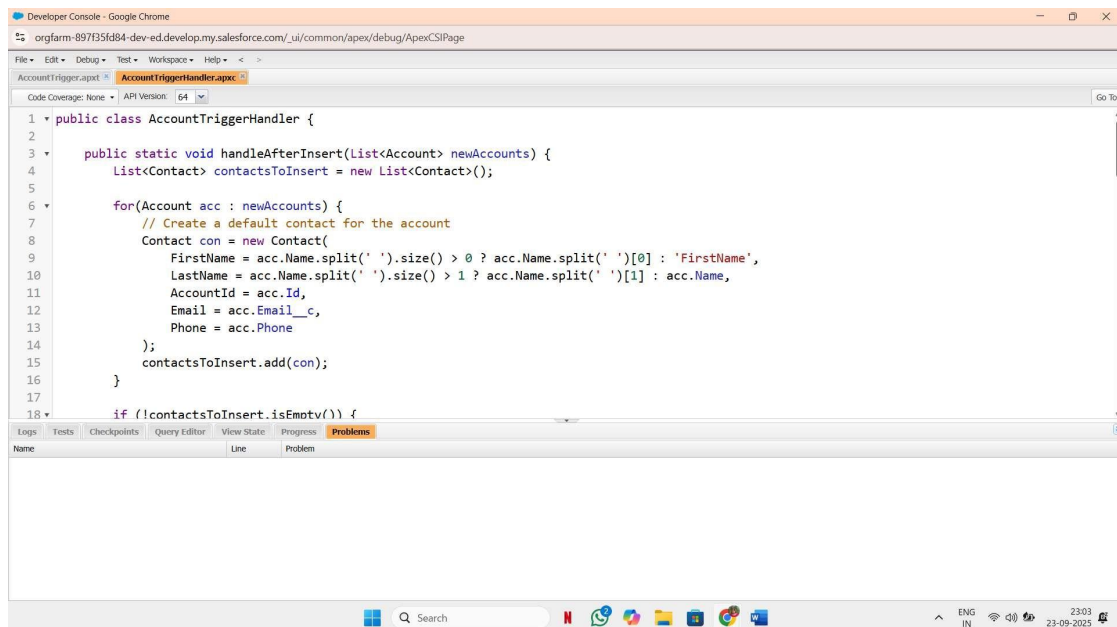
- **Whenever an Account is created or updated:**
 - Create a related Contact if it doesn't exist.
 - Update existing Contacts if the Account information changes.

AccountTriggerHandler:



The screenshot shows the Salesforce Developer Console with the file 'AccountTrigger.apex' open. The code is an Apex trigger that fires after insert and after update on the Account object. It calls the 'handleAfterInsert' and 'handleAfterUpdate' methods of the 'AccountTriggerHandler' class.

```
1 trigger AccountTrigger on Account (after insert, after update) {  
2     if (Trigger.isAfter) {  
3         if (Trigger.isInsert) {  
4             AccountTriggerHandler.handleAfterInsert(Trigger.new);  
5         }  
6         if (Trigger.isUpdate) {  
7             AccountTriggerHandler.handleAfterUpdate(Trigger.new, Trigger.oldMap);  
8         }  
9     }  
10 }
```



The screenshot shows the Salesforce Developer Console with the file 'AccountTriggerHandler.apex' open. The code is an Apex class that contains two static methods: 'handleAfterInsert' and 'handleAfterUpdate'. The 'handleAfterInsert' method creates a new Contact for each Account in the 'newAccounts' list. The 'handleAfterUpdate' method updates existing Contacts for each Account in the 'newAccounts' list.

```
1 public class AccountTriggerHandler {  
2  
3     public static void handleAfterInsert(List<Account> newAccounts) {  
4         List<Contact> contactsToInsert = new List<Contact>();  
5  
6         for(Account acc : newAccounts) {  
7             // Create a default contact for the account  
8             Contact con = new Contact(  
9                 FirstName = acc.Name.split(' ').size() > 0 ? acc.Name.split(' ')[0] : 'FirstName',  
10                LastName = acc.Name.split(' ').size() > 1 ? acc.Name.split(' ')[1] : acc.Name,  
11                AccountId = acc.Id,  
12                Email = acc.Email_c,  
13                Phone = acc.Phone  
14            );  
15            contactsToInsert.add(con);  
16        }  
17  
18        if (!contactsToInsert.isEmpty()) {  
19            // Insert the contacts  
20            insert contactsToInsert;  
21        }  
22    }  
23  
24    public static void handleAfterUpdate(List<Account> newAccounts, Map<Account, Account> oldMap) {  
25        List<Contact> contactsToUpdate = new List<Contact>();  
26  
27        for(Account acc : newAccounts) {  
28            // Update the contact  
29            Contact con = new Contact(  
30                FirstName = acc.Name.split(' ').size() > 0 ? acc.Name.split(' ')[0] : 'FirstName',  
31                LastName = acc.Name.split(' ').size() > 1 ? acc.Name.split(' ')[1] : acc.Name,  
32                AccountId = acc.Id,  
33                Email = acc.Email_c,  
34                Phone = acc.Phone  
35            );  
36            contactsToUpdate.add(con);  
37        }  
38  
39        if (!contactsToUpdate.isEmpty()) {  
40            // Update the contacts  
41            update contactsToUpdate;  
42        }  
43    }  
44 }
```

4. SOQL & SOSL

Use Cases:

- **SOQL:** Query orders placed in the last 30 days, products below reorder level, or customer order history.
- **SOSL:** Search products across multiple objects (Product Name, SKU, Description) when customer searches in the storefront.

5. Collections: List, Set, Map

- **List:** Store a list of orders to process in a batch.
- **Set:** Keep a set of unique SKUs from incoming orders to prevent duplicates.
- **Map:** Map Product Id → Stock Quantity to efficiently update inventory after an order.

We will be using these collections in our apex code to store the records of subjects into these so that we can use them for later in the code

6. Batch Apex

A way to process large volumes of data asynchronously in Salesforce by breaking it into smaller batches.

Use Cases:

- Process **large volume of orders** nightly to update inventory and financial records.
- Batch update product prices based on supplier cost changes.

7. Queueable Apex

A type of asynchronous Apex that allows chained jobs and complex processing in the background.

Use Cases:

- Perform **post-order processing** asynchronously (like sending confirmation emails and updating CRM systems).

8. Scheduled Apex

Apex code that runs at specific times or intervals, like daily, weekly, or monthly.

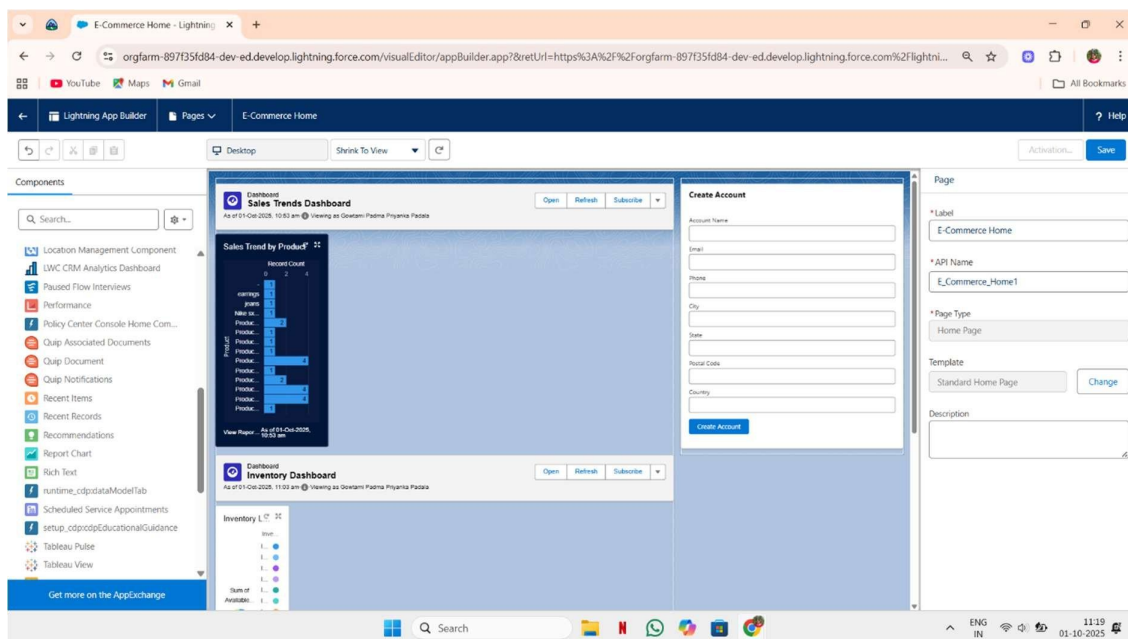
Use Cases:

- Run daily jobs to **check low-stock products** and notify the purchasing team.
- Weekly cleanup of old or canceled orders.

Phase 6: User Interface Development

1. Lightning App Builder

- **Definition:** A drag-and-drop tool to create custom Lightning pages for apps, record pages, and home pages.
- **Use Case:** Build a custom "E-Commerce Home" Home page that displays Lwc's and dashboards.



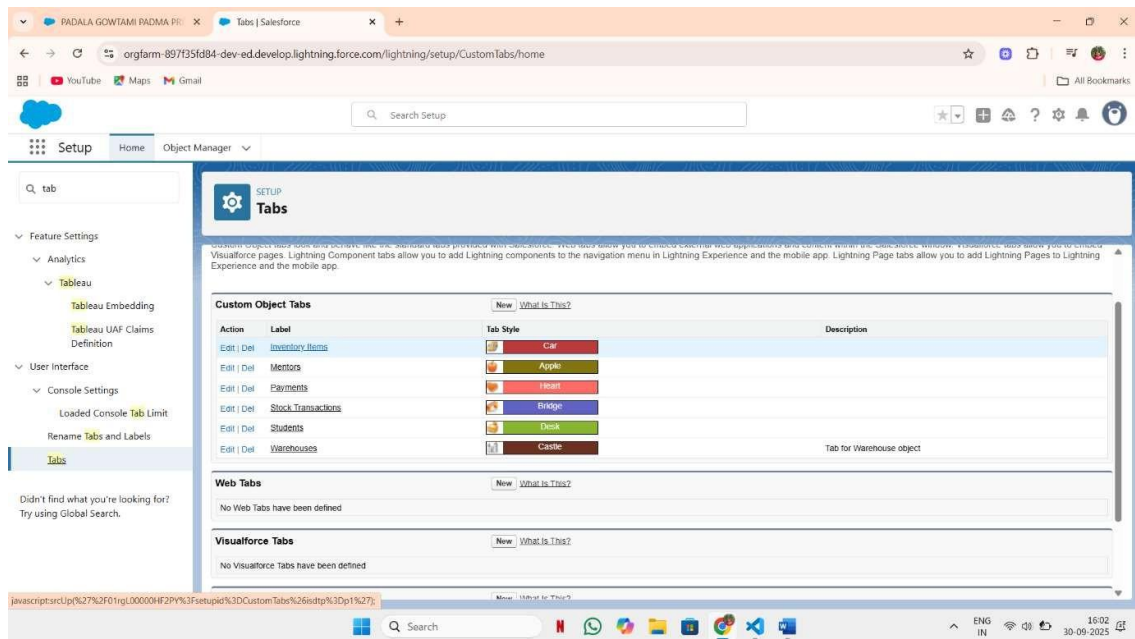
2. Record Pages

- **Definition:** Customizable layouts for object records, defining what users see when viewing a record.

3. Tabs

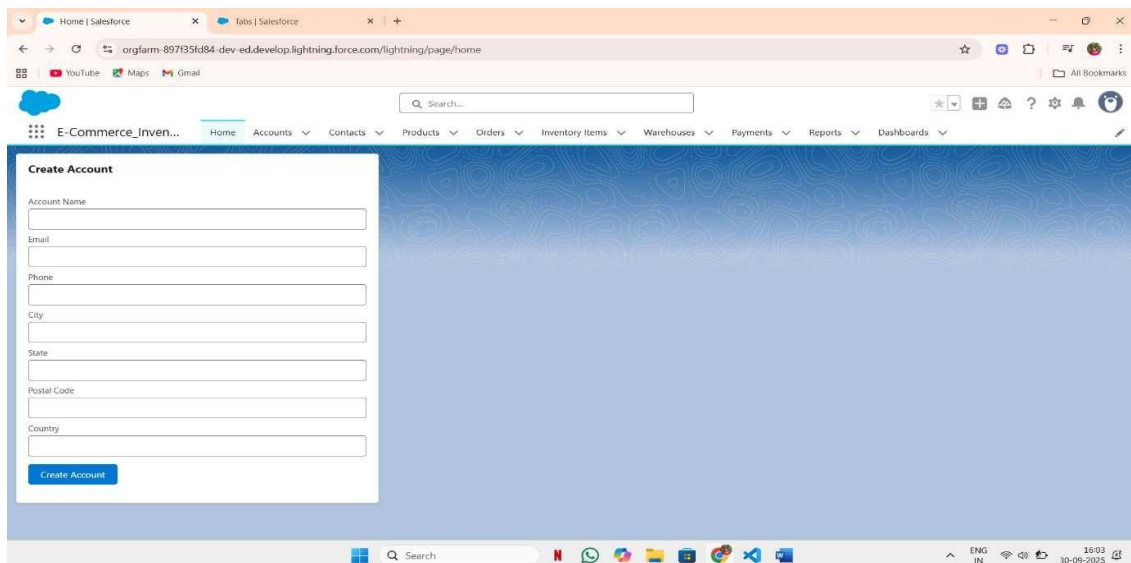
- **Definition:** Custom navigation elements to access objects or pages easily within an app.

- Create a **tab for "Inventory Items"** so warehouse staff can directly access stock details.



4. Home Page Layouts

- **Definition:** The main landing dashboard for users, tailored to roles.
- **Use Case:** A Sales rep Home Page showing dashboards like daily sales, pending high-value order and lwc.



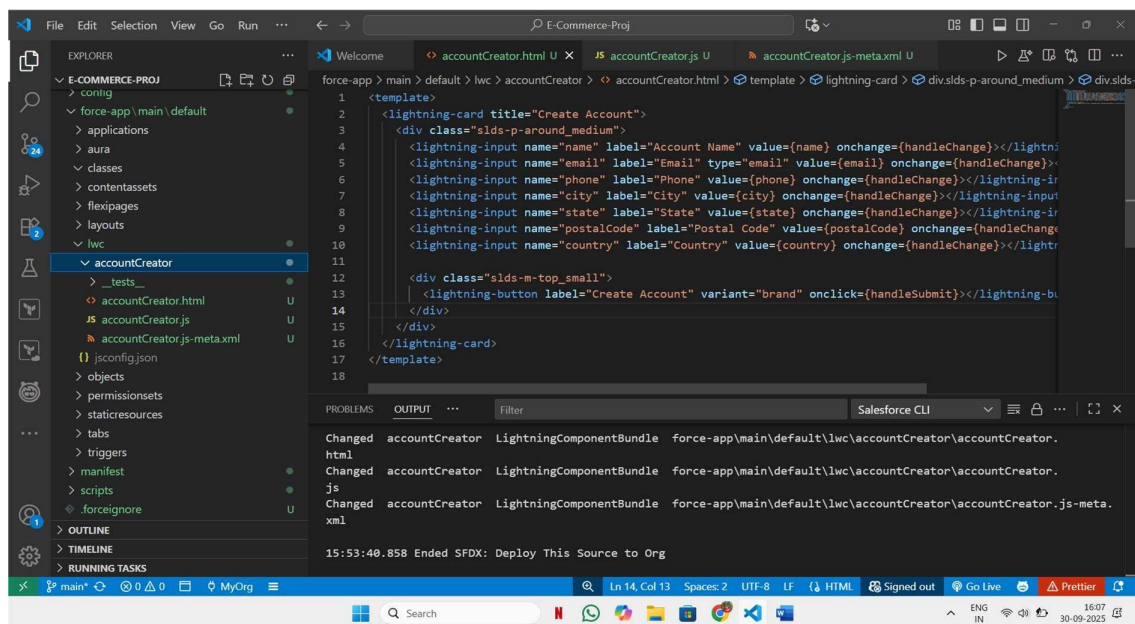
5. Utility Bar

- **Definition:** A customizable persistent bar in Lightning apps for quick access to tools.

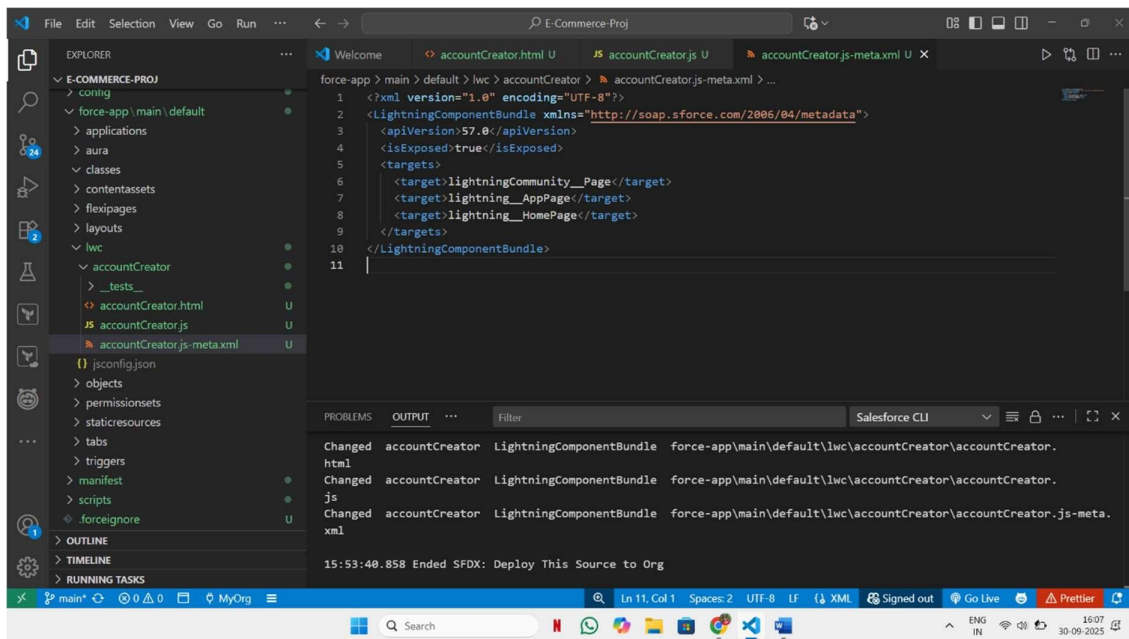
6. Lightning Web Components (LWC)

- **Definition:** Modern JavaScript-based components for building fast, reusable UI elements.
- **Use Case:** Create a “Real-Time Inventory Tracker LWC” to display live stock counts of popular products.
- **In this project I have created accountCreator LWC for the sales rep profile users.**
- **An LWC component will be created using visual studio code.**
- **Connect your org→create LWC using sfdx commands→Write code→Write the Controller code in salesforce Developer Console→Deploy LWC source to org from VSCode.**

➤ . HTML FILE:



➤ META.XML FILE:



➤ .javascript file:

```
import { LightningElement, track } from 'lwc';

import createAccount from '@salesforce/apex/AccountController.createAccount';

//import Email_c from '@salesforce/schema/Account.Email_c';

import { ShowToastEvent } from 'lightning/platformShowToastEvent';

import BillingCity from '@salesforce/schema/Account.BillingCity';

import BillingState from '@salesforce/schema/Account.BillingState';

import Name from '@salesforce/schema/Account.Name';

import Phone from '@salesforce/schema/Account.Phone';

import BillingCountry from '@salesforce/schema/Account.BillingCountry';

export default class AccountCreator extends LightningElement {

    @track name=""; email=""; phone=""; city=""; state=""; postalCode=""; country="";
```

```
handleChange(evt)
```

```
  { this[evt.target.name] =  
    evt.target.value;  
  }
```

```
handleSubmit()
```

```
  { const accountObj  
    = {  
      'objectType': 'Account',  
      Name: this.name,  
      Phone: this.phone,  
      Email__c: this.email,  
      BillingCity: this.city,  
      BillingPostalCode: this.postalCode,  
      BillingCountry: this.country,  
      BillingState: this.state  
    };  
  }
```

```
createAccount({ acc: accountObj })
```

```
  .then(account => {  
    this.dispatchEvent(new  
      ShowToastEvent({ title: 'Success',  
        message: 'Account created: ' + account.Id,  
        variant: 'success'
```

```

        });

        // clear inputs

        this.name=""; this.email=""; this.phone=""; this.city=""; this.state="";
        this.postalCode=""; this.country="";

    })

    .catch(error => {

        this.dispatchEvent(new

            ShowToastEvent({ title: 'Error',

                message: (error.body && error.body.message) ? error.body.message :
error.message,

                variant: 'error'

            });

    });

}

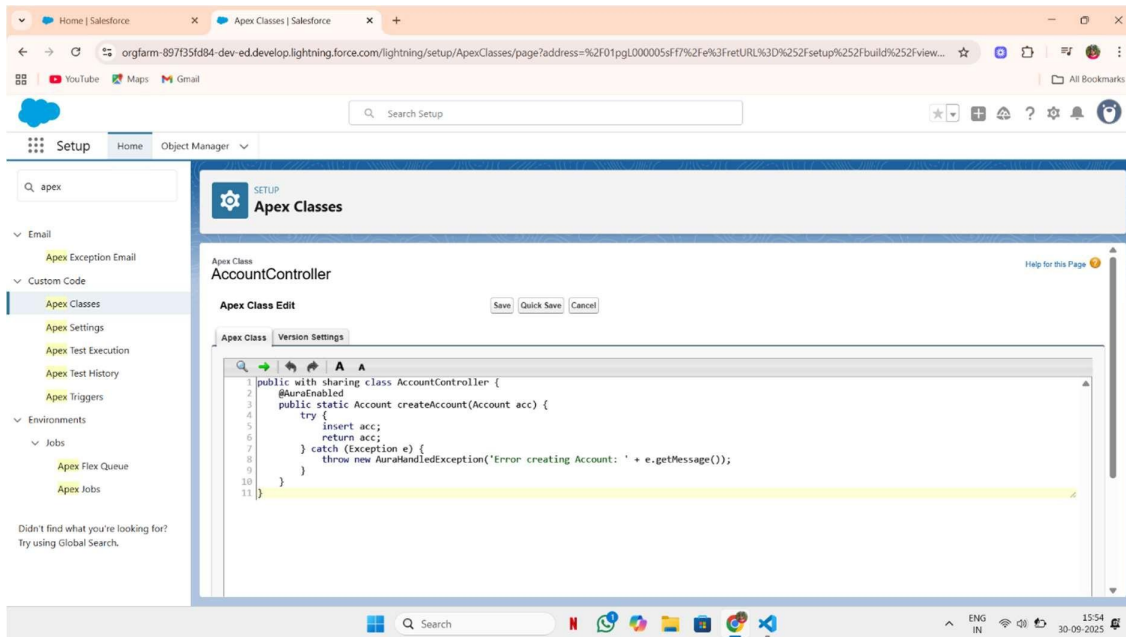
}

```

7. Apex with LWC

- **Definition:** Connecting LWCs to server-side Apex classes for business logic and database operations.
- **Use Case:** Build an LWC that **fetches live product availability** by calling an Apex method that checks warehouse stock levels.
- Build An apex controller that takes input from LWC and inserts accounts.

- I have used `@auraEnabled=true` so that the lwc component AccountCreator will find the AccountController.



8. Events in LWC

- **Definition:** Mechanism for communication between components (parent-child or sibling).
- **Use Case:** In an **Order Checkout Page**, when a user selects a product (child component), an event notifies the parent component to update the total price.

9. Wire Adapters

- **Definition:** A reactive mechanism to fetch Salesforce data declaratively into LWC using `@wire`.

10. Imperative Apex Calls

- **Definition:** Explicit, on-demand server calls from LWC to Apex for more control.

Phase 7: Integration & External Access

Key Tasks to Implement-Definitions:

1. Named Credentials

- Store external API endpoints and authentication details securely (e.g., payment gateway or shipping API).

2. External Services

- Use Schema definitions or OpenAPI specs to automatically create Apex actions for external REST services—ideal if you need live product-stock updates or shipping rates.

3. Web Services (REST/SOAP)

- Build or consume REST/SOAP APIs.
- Example: expose a REST API so your website or mobile app can fetch product availability in real time.

4. Callouts

- Make outbound requests to external systems (e.g., warehouse system or payment processor).

5. Platform Events & Change Data Capture

- Publish/subscribe to events for near-real-time updates—useful when inventory changes and your e-commerce frontend must react instantly.

6. Salesforce Connect

- Create external objects that read data live from external databases without storing it in Salesforce.

7. API Limits & Monitoring

- Track and plan API usage to avoid hitting governor limits.

8. OAuth & Authentication

- Implement secure authentication for external apps (e.g., enable OAuth for your customer portal or partner app).

9. Remote Site Settings

- Whitelist external endpoints you'll call from Apex.

➤ My Future Integrations to Implement

1. Frontend Application (Website / Mobile App)

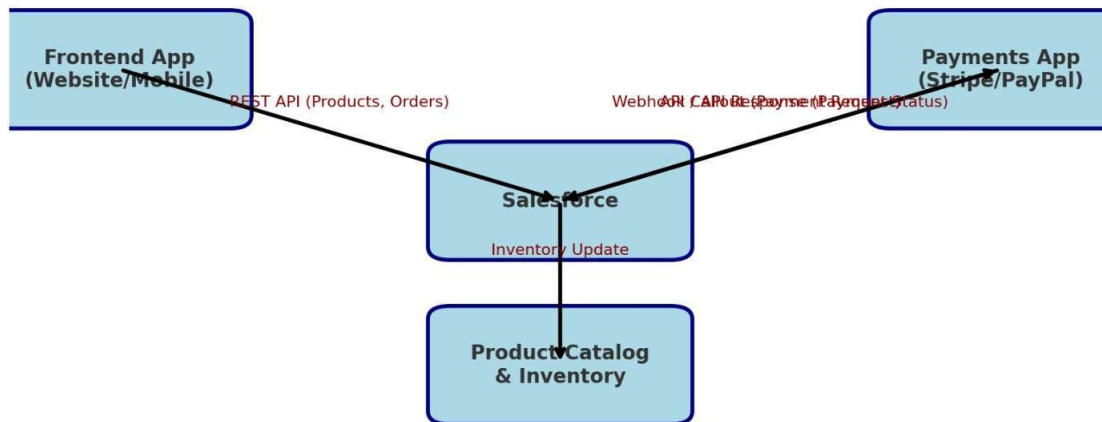
- Connect your e-commerce frontend (React, Angular, Flutter, etc.) with Salesforce via:
 - **Salesforce REST APIs** → to fetch product catalog, stock levels, customer orders.
 - **OAuth authentication** → so customers can securely log in.
 - **Change Data Capture (CDC)** → so the frontend gets *real-time updates* (e.g., stock sold out).

2. Payments Application

- Integrate with **payment gateways** like Stripe, Razorpay, or PayPal.
- Flow would be:
 1. Customer pays via frontend app.
 2. Payment API → confirms transaction.

3. Salesforce gets notified (via **API callout / webhook**) → updates **Order object** + reduces inventory.

Salesforce Integration with Frontend & Payments App

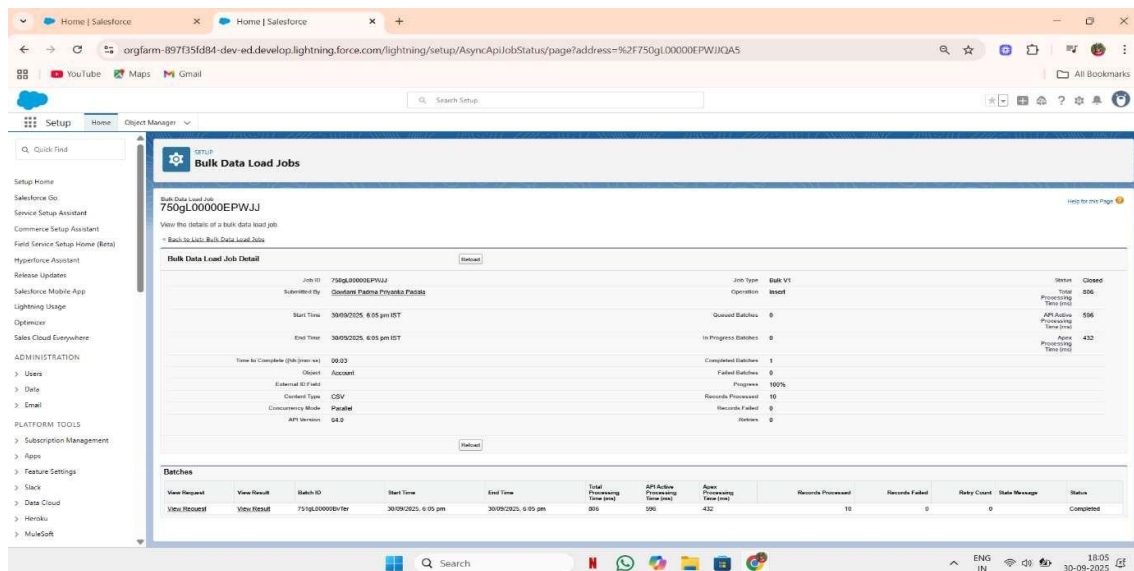
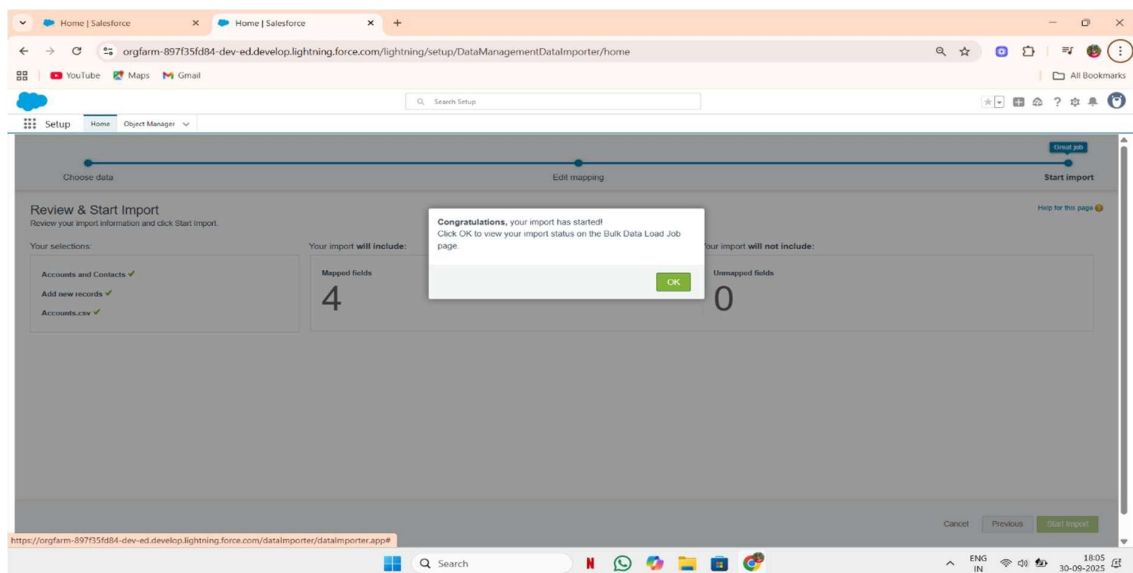


Phase 8: Data Management & Deployment

1. Data Import Wizard

Definition: A Salesforce tool used to import data for standard objects (like Accounts, Contacts, Leads) and custom objects in a simple way (up to 50,000 records).

→ For my project I will be using data Import wizard from inside the salesforce. I am importing data for Accounts, Inventory items, Warehouses, also payments.



Accounts

New This Week

15 items • Sorted by Account Name • Filtered by Created Date • Updated a minute ago

	Account Name	Account...	Billing State/Province	Phone	Type	Account Ow...
1	Barron PLC			+1-731-762-3078x43974		priya
2	Caldwell, Jackson and Guerrero			(135)209-3534x10007		priya
3	Cantrell-McLaughlin			(802)536-9625		priya
4	Cline, Taylor and Hall			859-527-5142x8747		priya
5	Gamble, Long and Cunningham			566.697.4196x8838		priya
6	Osborn-Iden			507.249.7573x7046		priya
7	PADALA GOWTAMI PADMA PRIYANKA		Andhra Pradesh	9640504034		priya
8	rsithi dev			775-555-5309		priya
9	Roberts, Gomez and Lopez			541.200.0882		priya
10	Rojas and Sons			+1-128-703-3402		priya
11	Rose-Torres			001-865-650-3063		priya
12	tamaja			775-555-5309		priya
13	tamaja reddy			9767164385		priya
14	test_student1334					priya
15	Wang Group			(452)395-7561x699		priya

Warehouse:

Warehouses | Salesforce

Setup

Bulk Data Load Jobs

Bulk Data Load Job Detail

Job ID	750gl00000ER1o3	Job Type	Bulk V1	Status	Closed
Submitted By	Gowtami Padma Priyanka Padala	Operation	Insert	Total Processing Time (min)	70
Start Time	30/09/2025, 11:21 pm IST	Queued Batches	0	API Active Processing Time (min)	33
End Time	30/09/2025, 11:21 pm IST	In Progress Batches	0	Apex Processing Time (min)	0
Time to Complete (hh:mm:ss)	00:00	Completed Batches	1	Progress	100%
Object	Warehouse	Failed Batches	0	Records Processed	5
External ID Field		Records Failed	0	Records Failed	0
Content Type	CSV	Retries	0	State Message	Completed
Concurrency Mode	Parallel				
API Version	64.0				

Batches

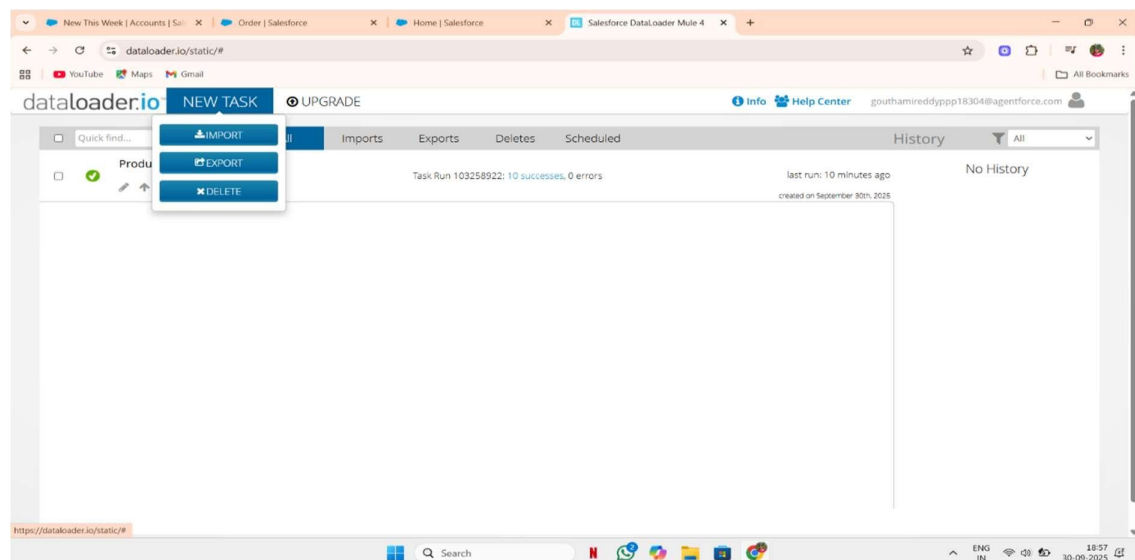
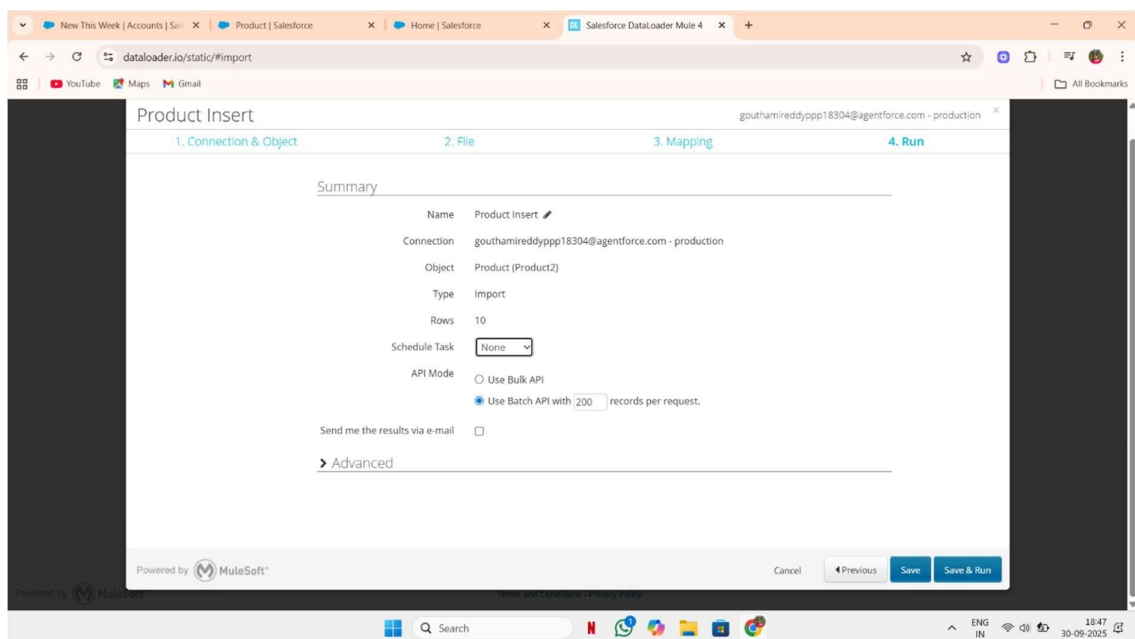
View Request	View Result	Batch ID	Start Time	End Time	Total Processing Time (min)	API Active Processing Time (min)	Apex Processing Time (min)	Records Processed	Records Failed	Retry Count	State Message	Status
View Request	View Result	751gl00000Bw4De	30/09/2025, 11:21 pm	30/09/2025, 11:21 pm	70	33	0	5	0	0		Completed

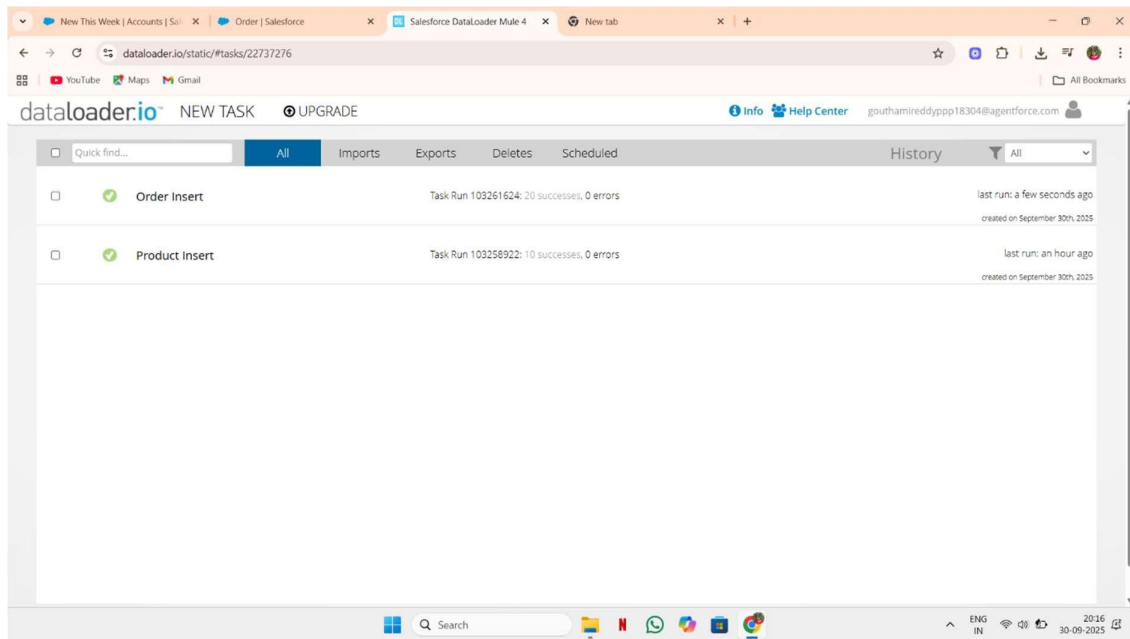
2. Data Loader

Definition: A client application for bulk import, update, delete, or export of Salesforce data (handles millions of records).

→ **Used for** Uploading **inventory stock levels**, updating **order statuses**, or deleting **old records** in bulk.

For products and orders I used dataloader.io to import the records into salesforce org

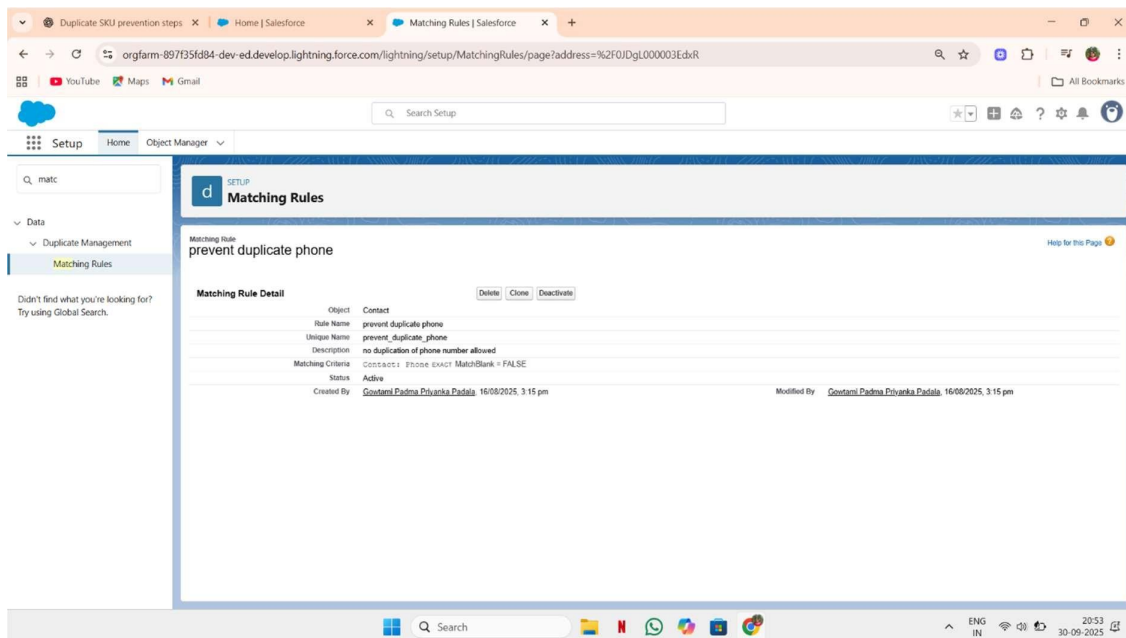


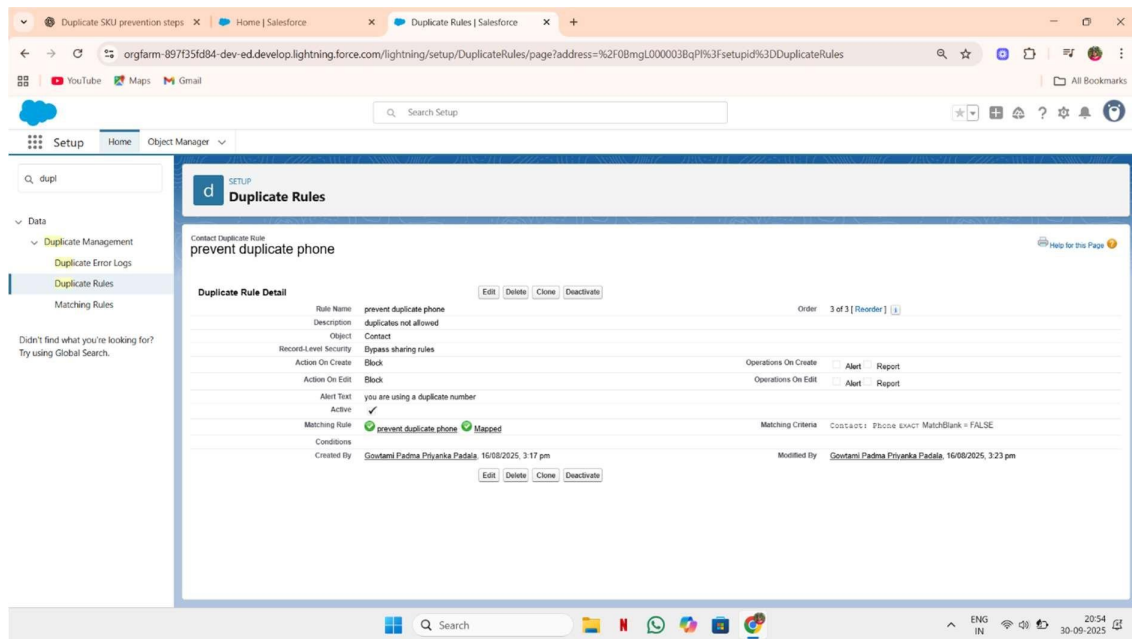


3. Duplicate Rules

Definition: Rules that prevent or warn users when duplicate records are being created in Salesforce.

Used for: Avoid having the **duplicate phone number** for contacts.

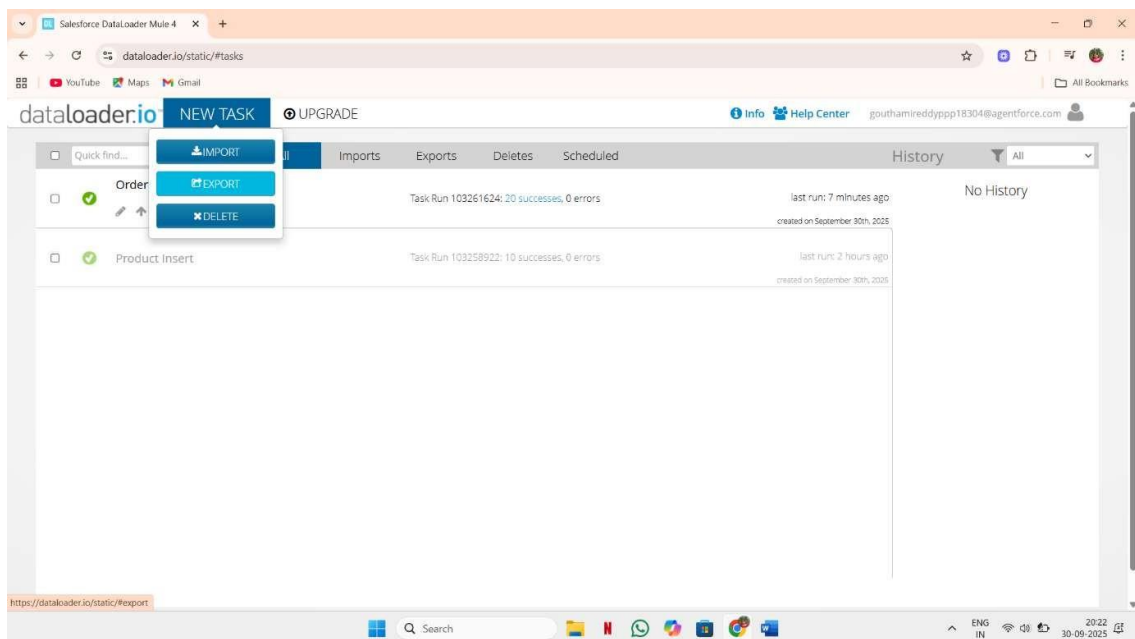




4. Data Export & Backup

Definition: Export Salesforce data into CSV files for backup or offline analysis (can be scheduled weekly/monthly).

Use Case: Keeping a **weekly backup of sales orders and inventory levels** for compliance and disaster recovery.



5. Change Sets

Definition: A Salesforce deployment tool that allows you to move metadata (custom objects, fields, Apex code, automation, etc.) from one Salesforce org (e.g., sandbox) to another (e.g., production).

Use Case: Deploying your **LWC product catalog page, Apex triggers for stock updates, and automation flows** from the sandbox into the live production org.

6. ANT Migration Tool

Definition: A command-line tool based on Apache ANT for deploying/retrieving metadata between Salesforce orgs. It's scriptable and works well for automation.

Use Case: Automating the **deployment of inventory objects, Apex triggers, and flows** from sandbox → UAT → production without manually using change sets.

7. VS Code & SFDX (Salesforce DX)

Definition: A modern CLI and IDE-based way to build, test, and deploy Salesforce apps with source-driven development.

Used in:

- Used for Version control in **LWC components** (GitHub integration).
- Automate **CI/CD pipelines** for e-commerce app updates.
- Easily rollback or deploy features.

Phase 9: Reporting, Dashboards & Security Review

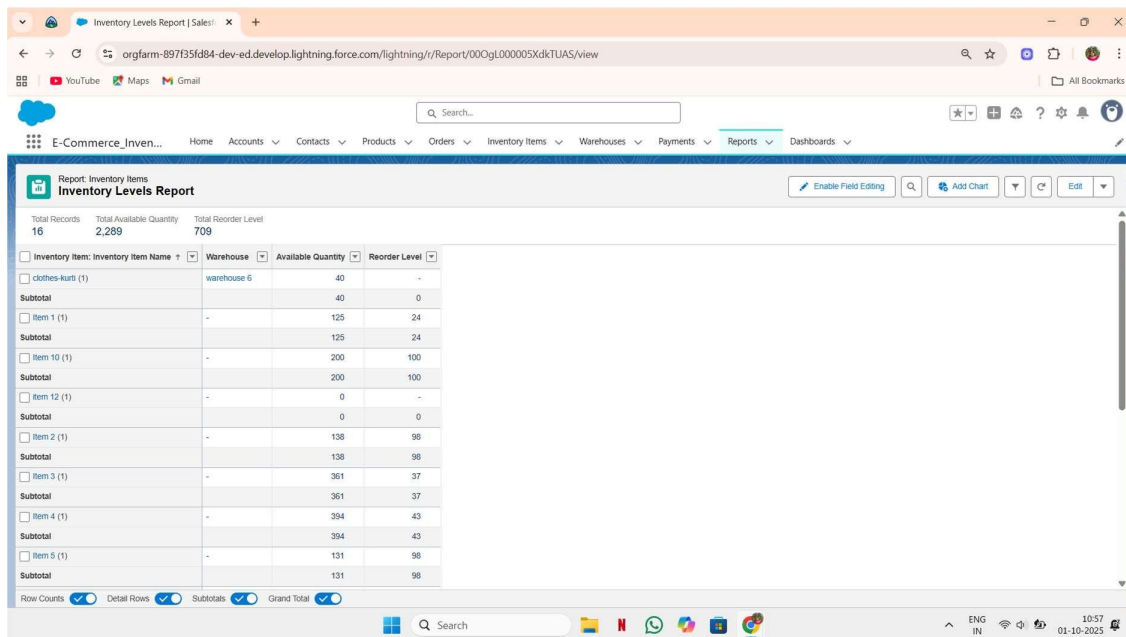
1. Reports (Tabular, Summary, Matrix, Joined)

Definition: Reports are structured lists of records in Salesforce that let you analyze your data. There are different formats:

- **Tabular Report:** Simple list of records. No grouping.

Use Case: **product inventory levels.**

- I have created an inventory level tabular report grouping on basis of inventory item summing up the total available quantity.



The screenshot displays a Salesforce report titled 'Inventory Levels Report' in a tabular format. The report is filtered by 'Warehouse' and shows data for various inventory items. The columns are 'Inventory Item: Inventory Item Name', 'Warehouse', 'Available Quantity', and 'Reorder Level'. The report includes subtotals for each item and a grand total at the bottom. The interface includes a search bar, navigation tabs, and a status bar at the bottom.

Inventory Item: Inventory Item Name	Warehouse	Available Quantity	Reorder Level
clothes-kurti (1)	warehouse 6	40	-
Subtotal		40	0
Item 1 (1)	-	125	24
Subtotal		125	24
Item 10 (1)	-	200	100
Subtotal		200	100
Item 12 (1)	-	0	-
Subtotal		0	0
Item 2 (1)	-	138	98
Subtotal		138	98
Item 3 (1)	-	361	37
Subtotal		361	37
Item 4 (1)	-	394	43
Subtotal		394	43
Item 5 (1)	-	131	98
Subtotal		131	98

sales per product category:

→ Created a sales trend report grouping on basis of products on the order object.

The screenshot shows a Salesforce report titled "Sales Trend by Product" with 25 total records. The report is a summary report grouped by "Product". The table displays columns for Product, Order Start Date, and Quantity. Subtotals are provided for each product group.

Product	Order Start Date	Quantity
- (1)	30/09/2025	-
Subtotal		
earnings (1)	07/10/2025	-
Subtotal		
jeans (1)	21/10/2025	3
Subtotal		
Nike sx Shoes (1)	30/09/2025	11
Subtotal		
Product 1 (2)	16/08/2025	4
	16/09/2025	4
Subtotal		
Product 10 (1)	27/08/2025	2
Subtotal		
Product 2 (1)	04/08/2025	2
Subtotal		
Product 3 (1)	10/09/2025	1

- **Summary Report:** Records grouped by field with subtotals.
Use Case: Total sales per account
- **Matrix Report:** Records grouped by rows and columns.
Use Case: Compare product sales across different regions and time periods.
- **Joined Report:** Combines multiple report blocks in a single report.
Use Case: Compare orders vs. inventory levels or sales vs. shipments.

2. Report Types

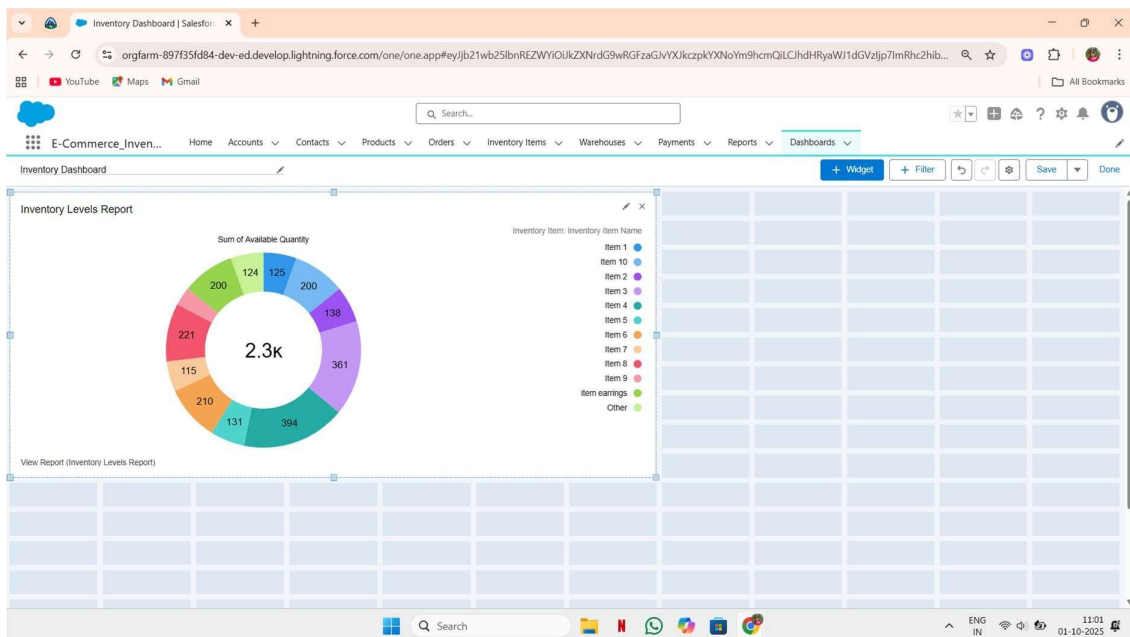
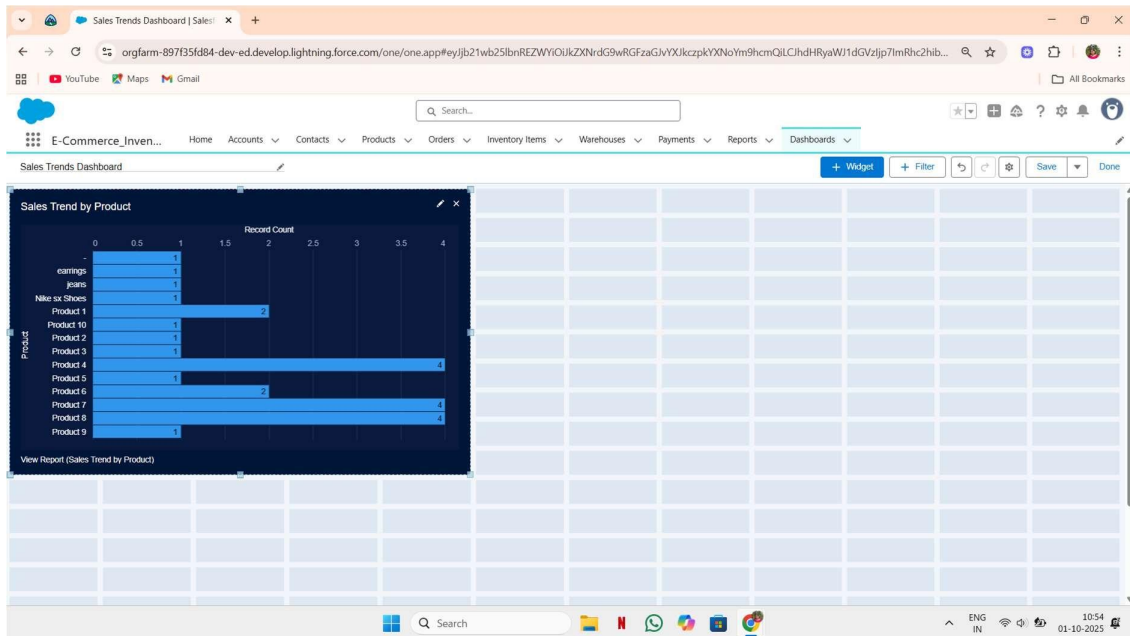
Definition: A template defining which objects and fields are available in a report.

Use Case: Custom report type to include Orders with Products and Warehouse details to track stock availability per order.

3. Dashboards

Definition: Visual display of multiple reports using charts, graphs, and metrics.

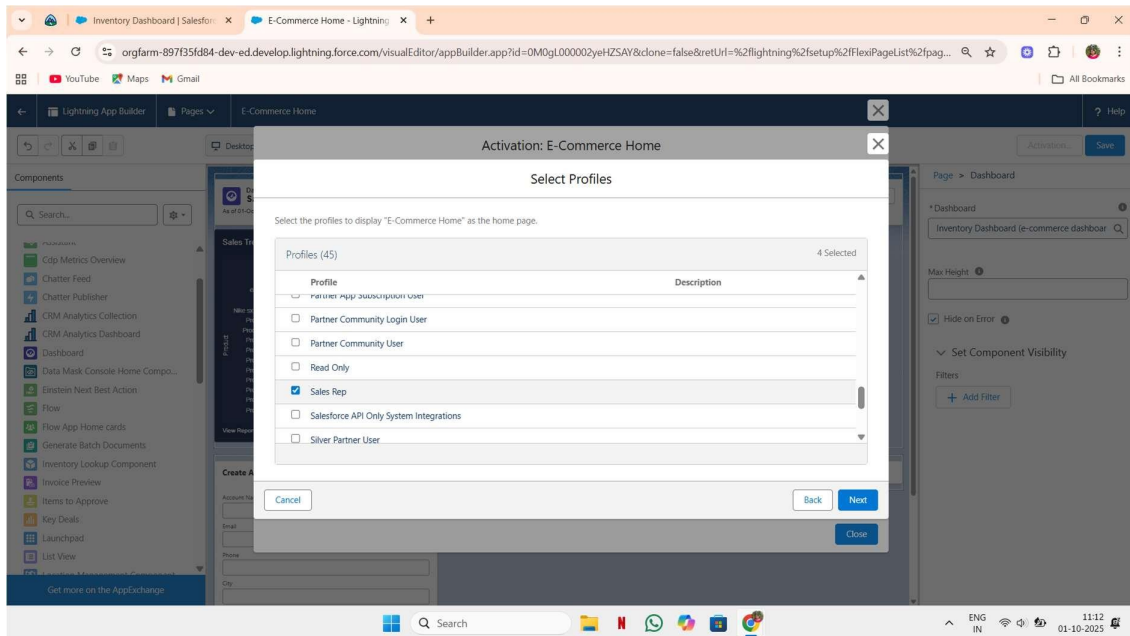
Use Case: Executive dashboard showing sales trends, top-selling products, and warehouse stock levels.



5. Sharing Settings

Definition: Control over who can view or edit records at the object and record level.

Use Case: Ensure warehouse managers can edit inventory, while sales reps can only view product availability.



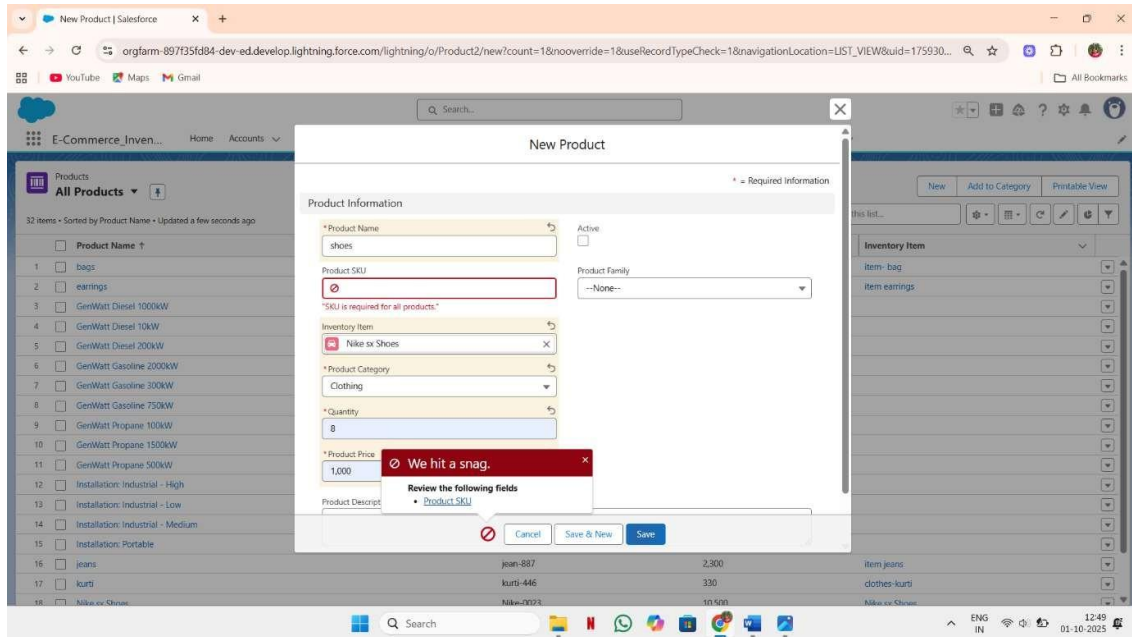
6. Field Level Security (FLS)

Definition: Restricts access to specific fields for different profiles or roles.

Use Case: Hide cost price from sales reps but visible to finance and inventory teams.

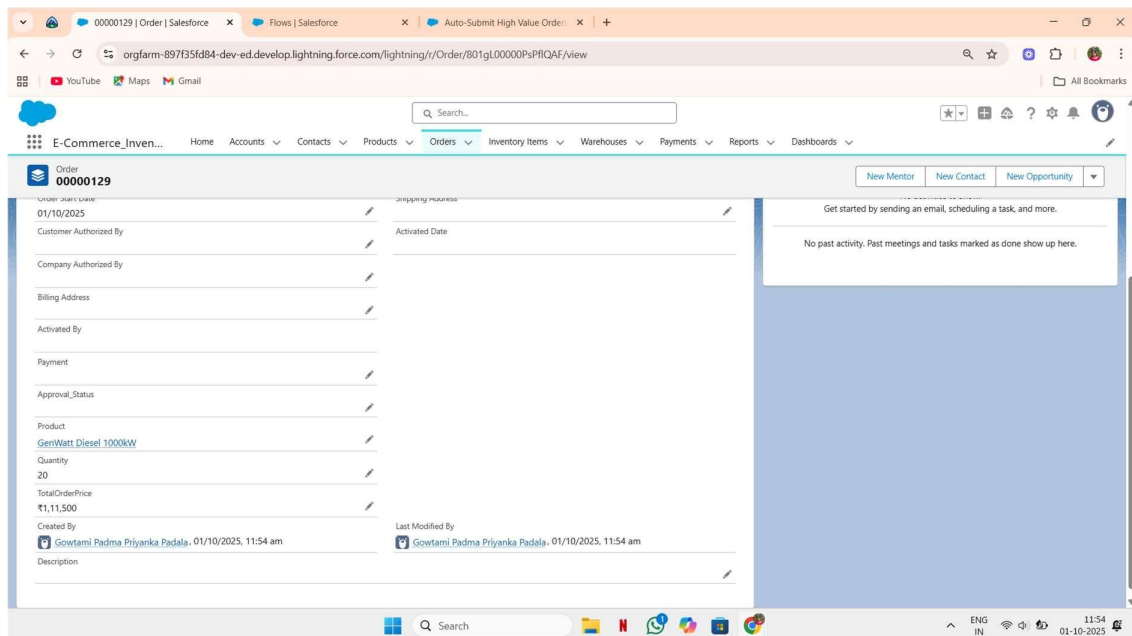
The screenshot displays the Salesforce Lightning Experience interface. On the left, the 'All Orders' list is visible, showing columns for Order Number, Account Name, and Order Date. The main area shows a 'New Order' modal form. The form includes fields for Billing City, Billing Zip/Postal Code, Billing State/Province, Payment, Approval Status, Product, Quantity, and Total Order Price. A red error message 'We hit a snag. Review the following fields' is displayed over the modal, indicating a validation error in the 'Quantity' field. The error message also lists 'Quantity' as the field to review. The background shows a list of orders with columns for Order Number, Account Name, and Order Date.

- SKU can't be null for products:



Approval Process:

- High Amount Order Approval:



00000129 | Order | Salesforce | Flows | Salesforce | Auto-Submit High Value Order

orgfarm-897f35fd84-dev-ed.develop.lightning.force.com/lightning/r/Order/801gL00000P8PIQAF/view

E-Commerce_Inven... Home Accounts Contacts Products Orders Inventory Items Warehouses Payments Reports Dashboards

Order 00000129

01/10/2025, 11:54 am Created. Govtami Padma Priyanka Padala

Notes & Attachments (0)

Payments (0)

Approval History (3)

Step Name	Date	Status	Assigned To
Step 1	01/10/2025, 11:54 am	Pending	Harika Nandra
Step 1	01/10/2025, 11:54 am	Pending	Tarun kumar Barri
Approval Request Submitted	01/10/2025, 11:54 am	Submitted	Govtami Padma Priyanka Padala

00000129 | Order | Salesforce | Auto-Submit High Value Order

orgfarm-897f35fd84-dev-ed.develop.lightning.force.com/lightning/r/Order/801gL00000P8PIQAF/view

E-Commerce_Inven... Home Accounts Contacts Products Orders Inventory Items Warehouses Payments Reports Dashboards

Order 00000129

01/10/2025, 11:54 am Created. Govtami Padma Priyanka Padala

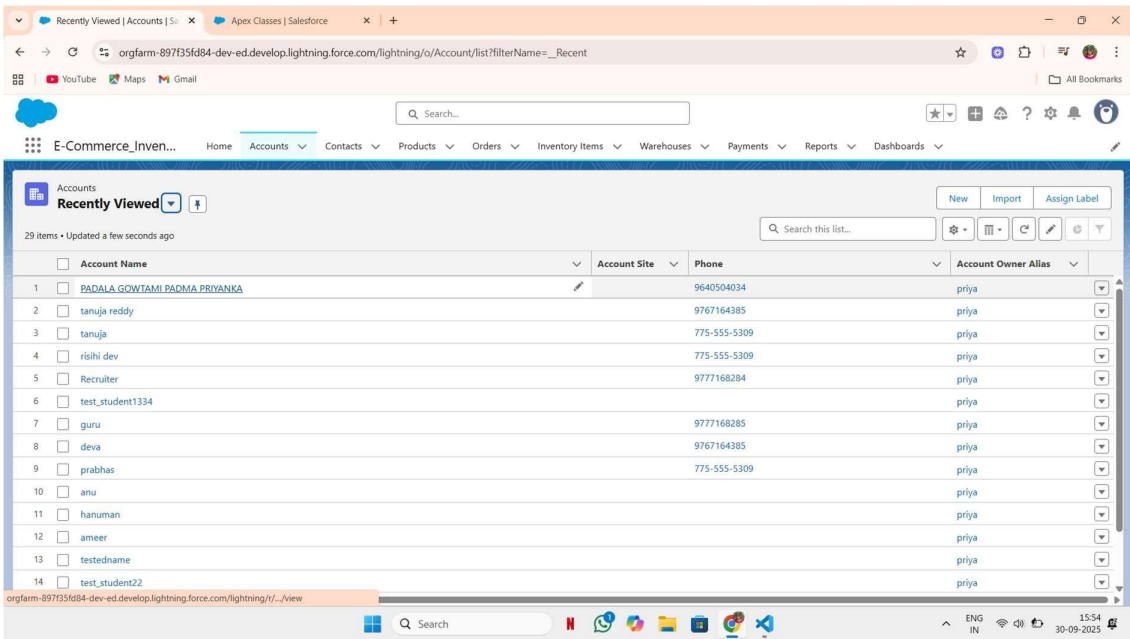
Notes & Attachments (0)

Payments (0)

Approval History (3)

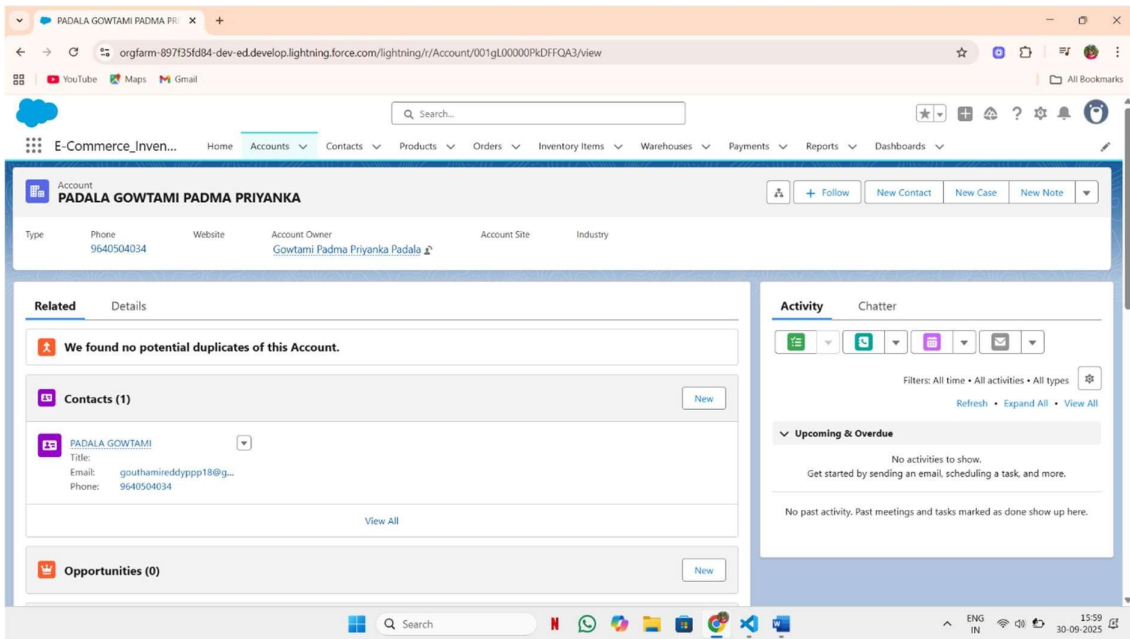
Step Name	Date	Status	Assigned To
Step 1	01/10/2025, 11:56 am	Approved	Tarun kumar Barri
Step 1	01/10/2025, 11:54 am	Approved	Harika Nandra
Approval Request Submitted	01/10/2025, 11:54 am	Submitted	Govtami Padma Priyanka Padala

Apex Trigger on Account To Create Contacts:



The screenshot shows the Salesforce interface with the 'Accounts' tab selected. The 'Recently Viewed' list displays 14 accounts. The table has columns for Account Name, Account Site, Phone, and Account Owner Alias. The first account is PADALA GOWTAMI PADMA PRIYANKA, with phone number 9640504034 and owner priya. The other accounts are tanuja reddy, tanuja, risihi dev, Recruiter, test_student1334, guru, deva, prabhas, anu, hanuman, ameer, testedname, and test_student22, all with phone numbers and owner priya.

	Account Name	Account Site	Phone	Account Owner Alias
1	PADALA GOWTAMI PADMA PRIYANKA		9640504034	priya
2	tanuja reddy		9767164385	priya
3	tanuja		775-555-5309	priya
4	risihi dev		775-555-5309	priya
5	Recruiter		9777168284	priya
6	test_student1334			priya
7	guru		9777168285	priya
8	deva		9767164385	priya
9	prabhas		775-555-5309	priya
10	anu			priya
11	hanuman			priya
12	ameer			priya
13	testedname			priya
14	test_student22			priya



The screenshot shows the detail view for the account PADALA GOWTAMI PADMA PRIYANKA. The account type is 'Individual', phone is 9640504034, and the owner is Gowtami Padma Priyanka Padala. The 'Related' section shows no potential duplicates and one contact, PADALA GOWTAMI, with email gouthamireddyppp18@g... and phone 9640504034. The 'Activity' section shows no upcoming or overdue activities.

Account: PADALA GOWTAMI PADMA PRIYANKA

Type: Individual **Phone:** 9640504034 **Website:** **Account Owner:** Gowtami Padma Priyanka Padala

Related:

- We found no potential duplicates of this Account.**
- Contacts (1)**
 - PADALA GOWTAMI**
 - Title:
 - Email: gouthamireddyppp18@g...
 - Phone: 9640504034
- Opportunities (0)**

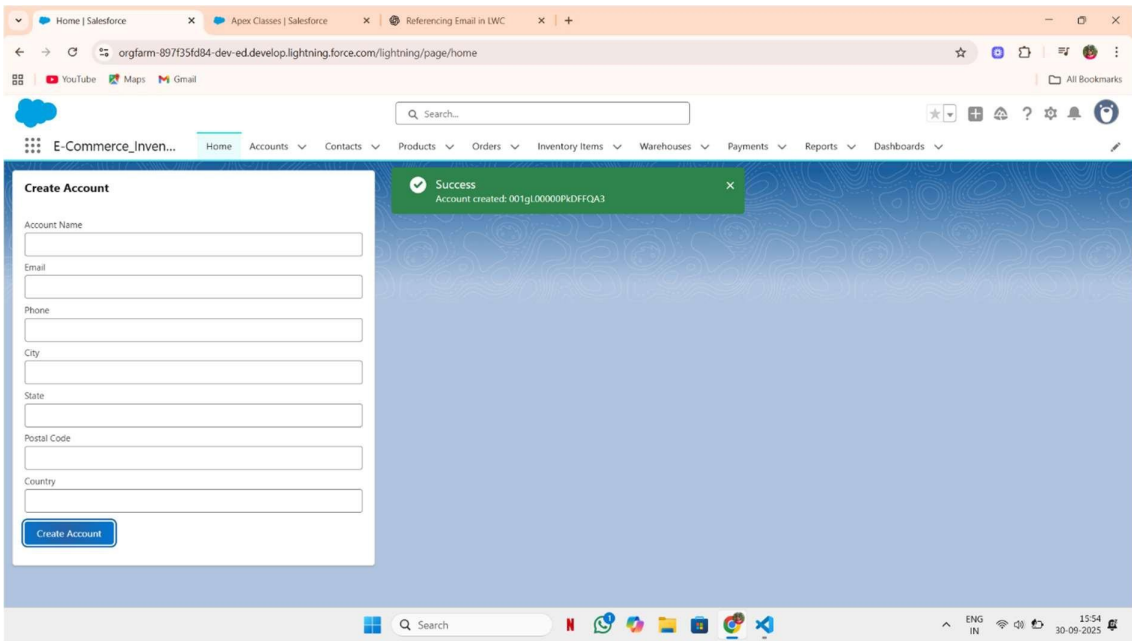
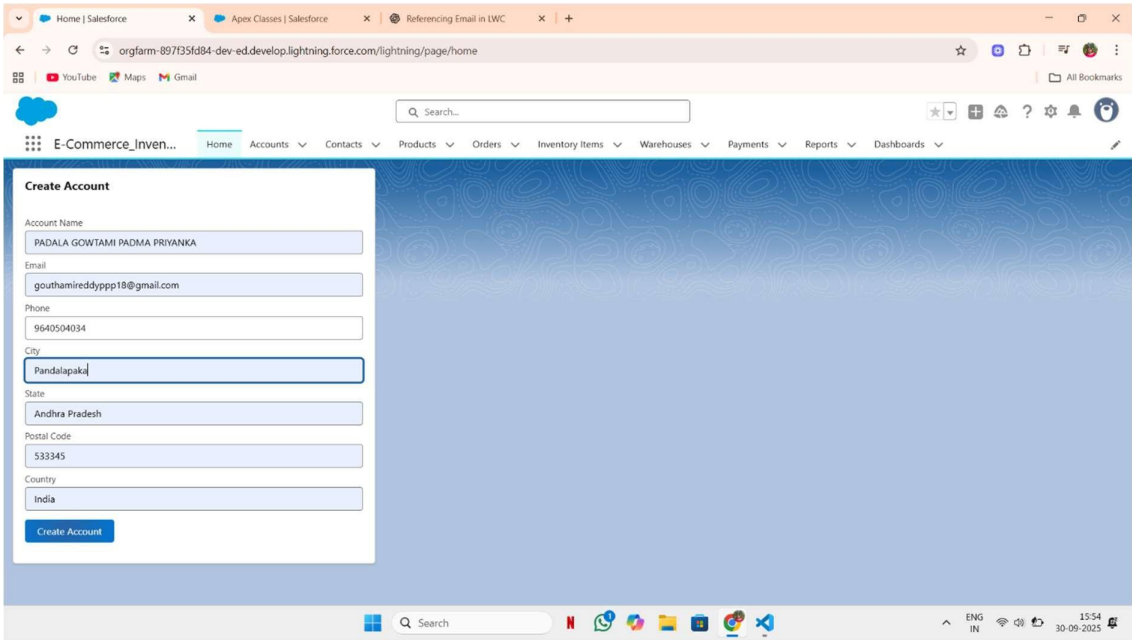
Activity:

Upcoming & Overdue

No activities to show.
Get started by sending an email, scheduling a task, and more.

No past activity. Past meetings and tasks marked as done show up here.

LWC Component to Create Account:



	Account Name	Account Site	Phone	Account Owner Alias
1	PADALA GOWTAMI PADMA PRIYANKA		9640504034	priya
2	tanuja reddy		9767164385	priya
3	tanuja		775-555-5309	priya
4	risih dev		775-555-5309	priya
5	Recruiter		9777168284	priya
6	test_student1334			priya
7	guru		9777168285	priya
8	deva		9767164385	priya
9	prabhas		775-555-5309	priya
10	anu			priya
11	hanuman			priya
12	ameer			priya
13	testedname			priya
14	test_student22			priya

Lightning App Builder-Home page for E-commerce_Inventory app

